UNIVERSIDAD ANDRÉS BELLO
Facultad de Ingeniería

Entrega 2

Backend seguro

Chocomanía: Sistema de Ventas Online
Caso 19 adaptado a una chocolatería

Asignatura	Desarrollo Web y Móvil, sección 8750
Docente	Ernesto Vivanco
Integrantes	Florencia Pacheco · Karla Muñoz
Versión del frontend	Entrega 1 (carpeta `frontend/`), conectada al backend en esta entrega
Repositorio	github.com/florenciapachecopardo-star/desarollo-web-unab
Fecha	9 de noviembre de 2026

Índice del cuerpo

1. Resumen
2. M1. Matriz de servicios y contratos
3. M2. Modelo del dominio
4. M3. Matriz de sistemas y componentes
5. M4. Stakeholders y matriz de permisos
6. M5. Gateway y fronteras de confianza
7. M6. Matriz de rutas e inventario de secretos
8. M7. Datos sensibles y pruebas de seguridad
9. M8. GraphQL Gateway
10. M9. Caracterización de carga y decisiones de arquitectura
11. M10. Backend completo e integración verificable
12. Matriz de requisitos
13. Brechas declaradas
14. Anexos y evidencias

Resumen

Chocomanía es la tienda online de una chocolatería pequeña. En esta entrega se construyó el backend detrás del front de la Entrega 1: un API Gateway como única puerta de entrada, seis servicios internos en una red privada (auth, clients, inventory, orders, payments y graphql-gateway), Vault para los secretos del gateway y MongoDB como base. El cliente puede comprar de punta a punta desde el navegador: registrarse y verificar su correo, ver el catálogo, iniciar sesión, crear el pedido, pagarlo, ver la boleta y revisar sus pedidos por GraphQL.

Lo que cruza todo el sistema:

- **Una sola puerta:** solo el gateway publica un puerto (8000) y solo deja pasar las rutas de su lista blanca.
- **Correlación:** cada solicitud lleva un `X-Request-ID` que aparece en la respuesta, en los logs JSON de cada servicio y en lo que se guarda en la base.
- **El servidor decide:** precios, totales, montos, estados, cajero y boleta los calcula el backend; lo que mande el navegador se ignora.
- **Un secreto por integración** y propiedad de los recursos revisada en cada servicio, no solo en el gateway.
- **Idempotencia** en pedidos y pagos (`Idempotency-Key`).

M1. Matriz de servicios y contratos

Documento: `m1-servicios.md`

13 requisitos (R01 a R13) tomados del Caso 19, 9 flujos (F01 a F09) y 19 servicios (S01 a S19). Cada servicio tiene su requisito, su flujo, la pantalla que lo usa, el acceso, la clasificación (consulta, transaccional, integración, agregación o analítica), el estado de implementación y un ejemplo JSON real tomado del seed o de las evidencias. Las reglas comunes cubren el formato único de error, la validación (422), la correlación, la identidad, el recurso ajeno (404), la idempotencia y qué pasa si un servicio cae.

Los 19 servicios están implementados, más S09V (verificación del correo). S09, S09V y S10 se agregaron el 9 de noviembre con el servicio clients.

M2. Modelo del dominio

Documentos: `m2-dominio.md` y `uml-dominio.drawio`

Diagrama de clases con Usuario, Rol, Cliente, Producto, Pedido, LineaPedido, Boleta y Pago, sus multiplicidades (por ejemplo, Pedido y LineaPedido 1 a 1..* por composición, Pedido y Pago 1 a 0..*), 13 reglas del dominio (D01 a D13) y la máquina de estados del pedido. Confirmado con el código de orders y payments. Cliente quedó implementado en el servicio clients (perfil con RUN único); las credenciales siguen en auth.

M3. Matriz de sistemas y componentes

Documentos: `m3-sistemas.md` y `componentes.drawio`

Cada sistema con su responsabilidad, sus datos y su tipo de dependencia (real, doble de prueba o no implementado). Despacho y boletas viven dentro de orders y el proveedor de pago es un doble de prueba que respeta el contrato. Incluye las reglas entre sistemas, la secuencia completa de una compra (crear, pagar y ver mis pedidos) ligada a sus evidencias, y los límites conocidos (por ejemplo, la devolución de stock sin reintento).

M4. Stakeholders y matriz de permisos

Documentos: `m4-stakeholders.md` y `m4-permisos.md`

Registro de 11 stakeholders con su interés e influencia. Matriz de permisos por servicio y rol (Anónimo, CLIENTE, ADMIN, DUENO, CAJERO, DESPACHO), las operaciones internas que no expone el gateway (reservas de stock, marcar pagado, lectura para GraphQL) con el secreto que exige cada una, y las reglas que salen de la matriz.

M5. Gateway y fronteras de confianza

Documentos: `m5-gateway.md` y `uml-gateway.drawio`

Por qué un solo gateway, el UML de sus clases (incluidas GraphQLGateway, Schema y Resolver), la clasificación de rutas (anónimas, autenticadas, por rol, integración y front estático) y las fronteras de confianza entre navegador, gateway, servicios internos, graphql-gateway y proveedor. Muestra el recorrido de una solicitud autorizada (S11, `crear-pedido-0001`, en cuatro servicios) y el de una denegada (S03 como CLIENTE).

M6. Matriz de rutas e inventario de secretos

Documento: `m6-rutas-secretos.md`

Matriz de todas las rutas públicas con método, acceso, servicio destino y controles; inventario de los secretos (uno por integración, quién lo tiene y dónde vive); qué hacer si se filtra una clave; qué pasa si Vault no responde (503 `SECRETS_UNAVAILABLE`, caché de 60 s); y la revisión de secretos en el repo, capturas y logs.

M7. Datos sensibles y pruebas de seguridad

Documento: `m7-datos-sensibles.md`

Cinco niveles de clasificación y una matriz de datos con finalidad, custodio, roles, protección, retención y estado real en la U2. Chocomanía no custodia secretos del cliente (no recibe datos de tarjeta). Incluye la minimización, el recorrido de la dirección de despacho, la verificación de correo simulada, las 6 pruebas de seguridad (P01 a P06, todas aprobadas con pytest en E07) y los riesgos de datos que quedan abiertos.

M8. GraphQL Gateway

Documento: `m8-graphql.md`

GraphQL Gateway privado (Strawberry) detrás del API Gateway, con las queries `misPedidos` y `pedido`. Junta orders e inventory con DataLoader (una sola llamada a inventory por consulta), limita profundidad (6), alias (10), tokens (1000) y paginación (20), autoriza por sesión, rol, recurso y campo (`stock` solo ADMIN), y devuelve errores parciales: si inventory cae, los pedidos igual se ven y `producto` viene null con `UPSTREAM_UNAVAILABLE`. Lo usa `mis-pedidos.html`.

M9. Caracterización de carga y decisiones de arquitectura

Documentos: `m9-carga.md`, `adr-grpc.md` y `adr-analitica.md`

Carga proyectada de un local pequeño: hasta 500 pedidos y 10.000 consultas al catálogo por día en peak, menos de 25 solicitudes por segundo. Umbrales fijados antes de medir (p95 del catálogo menor a 500 ms, de crear pedido menor a 1.000 ms, errores menores a 1 %). Script k6 del catálogo en `tests/carga/catalogo.js`.

Medición (E08): la primera ejecución no cumplió los umbrales (p95 de 3,2 s y 22 solicitudes por segundo) porque el gateway creaba un cliente HTTP por solicitud (INC06). Con el arreglo, el catálogo cumple con holgura: p95 de 19 ms, 65 solicitudes por segundo y 0 % de errores. Los umbrales no se cambiaron. Una tercera ejecución, con `docker stats`, midió CPU y memoria: gateway e inventory llegan a 87 % y 93 % de un núcleo, con 57,5 MiB y 92,9 MiB como máximo y sin reinicios.

- **ADR gRPC:** no se usa. Con esta carga, HTTP con JSON entre servicios alcanza y es más fácil de depurar y correlacionar.
- **ADR analítica:** el reporte de ventas es un endpoint dentro de orders (S19), no un servicio aparte.

M10. Backend completo e integración verificable

Qué está integrado

Componente	Qué hace	Evidencia
gateway	Lista blanca, sesión por cookie, identidad hacia los servicios, `X-Request-ID`, logs JSON, errores con formato único, secretos desde Vault, front estático en el mismo origen, un solo cliente HTTP (INC06)	E01, E07, E08
auth	Usuarios y sesiones en Mongo, bcrypt costo 12, token guardado como hash, TTL de 900 s	E02 a E06 (`/introspect` en cada traza)
inventory	Catálogo, stock, reservas y devoluciones internas, lote para GraphQL	E01, E02, E05, E10
orders	Pedidos con idempotencia, precio del servidor, cajero virtual, anulación con motivo, cola de despacho, estados, boleta y reporte de ventas	E02, E03, E10
payments	Pago con idempotencia, proveedor simulado (inmediato o aviso), reconciliación si orders cae	E03, E04
clients	Registro con RUN validado en el servidor, verificación del correo (simulada), perfil propio con RUN enmascarado; la credencial la crea auth con bcrypt	E07 (`test_clientes.py`)
graphql-gateway	`misPedidos` y `pedido` con DataLoader, límites y errores parciales	E05, E06
front	Registro, verificación, catálogo, carrito, login, checkout, pago, boleta y mis pedidos contra el backend real	E09 (escritorio, celular y error parcial)

Cómo verificarlo

1. Copiar `.env.example` a `.env` y completar los valores (en Windows, `scripts/generar-env.py` lo crea con secretos aleatorios).
2. `docker compose up -d --build` y esperar a que los servicios queden `healthy`.

3. `docker compose run --rm seed --reset` (o `scripts/reset.sh`) para cargar el seed (6 usuarios y 5 productos con ID fijos).
4. Abrir `http://localhost:8000`, iniciar sesión con `cliente@correo.cl` y comprar un producto.
5. Copiar el `X-Request-ID` de cualquier respuesta y ejecutar `scripts/logs.sh <request_id>` para ver la traza en todos los servicios.
6. Revisar la base con `docker compose run --rm check pedido PED-0003`.
7. Correr las pruebas automáticas con `docker compose run --rm pruebas` (22 pruebas, E07; esperar 1 minuto si se repite) y la carga con `docker compose run --rm k6` (E08), cada una sobre la base reseteada.

Lo que no está en la U2

El panel interno (mantenedor, cola de despacho y reporte no tienen pantalla, aunque sus servicios funcionan), el envío de la boleta por correo y las pruebas con Playwright. Ver Brechas declaradas.

Historial

Cada cambio está en `bitacora.md` (INCR-00 en adelante) y las fallas encontradas en `incidencias.md`.

Matriz de requisitos

Documento: `matriz-requisitos.md`

Une cada requisito con su pantalla, servicio, prueba, resultado y evidencia. R01 a R05, R07, R09, R12 y R13 tienen evidencia aprobada; R08 y R10 en parte (sin correo y sin mantenedor de usuarios y clientes); R06 y R11 siguen como brecha (registro por el administrador y mensajería real). Los servicios sin pantalla (productos como ADMIN, cola de despacho y reporte) se probaron por API (E10).

Brechas declaradas

Brecha	Motivo	Módulo donde se declara
R06 (registro hecho por el administrador) y R10 para usuarios y clientes	clients ya existe, pero falta la pantalla y la ruta de ADMIN	M1
Verificación del correo simulada, sin API de mensajería (R11)	Recorte acordado	M7
Boleta sin envío por correo (R08 parcial)	Sin servicio de mensajería	M1, M2
Panel interno sin pantallas: mantenedor de productos, cola de despacho y reporte de ventas solo por API	Recorte por tiempo	M1, M10
Proveedor de pago simulado (doble de prueba, no Webpay)	Recorte acordado	M3
Solo el gateway lee secretos de Vault; Vault en modo dev	Entorno local	M6
Todos los servicios usan el usuario raíz de Mongo	Entorno local	M6, M7
Cookie sin `secure` en local	No hay HTTPS en local	M6, M7
Sin pruebas con Playwright	Pasa a la U3; en la U2, evidencia manual	Protocolo de pruebas
Stock descontado sin pedido si inventory responde después del plazo de orders	Requiere reservas con ID propio	M3
GraphQL: `stock` no lo puede ver nadie (las queries son solo de CLIENTE), lote máximo de 50 productos e introspección activa	Decisiones de alcance	M8
Cuenta de cliente de prueba visible en el login (las del personal se sacaron del front)	Facilita la demo; es ficticia	M7
Solo se sometió a carga el catálogo; orders, payments y graphql-gateway todavía crean un cliente HTTP por solicitud, y gateway e inventory llegan cerca de un núcleo con 100 usuarios	Alcance de la medición	M9

Brecha	Motivo	Módulo donde se declara
DESPACHO se entera del estado de un pedido sin pagar al intentar moverlo (409 en vez de 404, INC07)	Encontrado al revisar E10	Incidencias

Anexos y evidencias

- Índice de evidencias: `../evidencias/indice.md` (E01 a E10, cada una con su ficha).
- Protocolo de pruebas: `protocolo-pruebas.md`.
- Incidencias: `incidencias.md`.
- Bitácora de incrementos: `bitacora.md`.
- Reparto de roles: `reparto-roles-entrega-2-v2.md`.

> Los documentos completos de cada módulo (matrices, contratos, ejemplos JSON y diagramas) están en los anexos de este PDF, en el mismo orden.

M1. Matriz de servicios y contratos

Caso: 19, Sistema de Ventas Online, adaptado a Chocomanía (chocolatería) **Fuente de la lista de servicios:** reparto de roles v2, sección 5. **Contratos confirmados con el código de `main` (3ba032e):** S01 a S08 y S11 a S19, comparados con `gateway.py`, `auth`, `inventory`, `orders`, `payments`, `graphql-gateway` y `frontend/js`. S09, S09V y S10 se confirmaron con `backend/clients/main.py` (agregados el 9 de noviembre). **Datos de los ejemplos:** salen de `scripts/seed.py` y de las evidencias de `evidencias/u2/`.

1. Identificadores

Prefijo	Qué es	Regla
R	Requisito funcional del Caso 19. R01 = RF1, R02 = RF2, y así hasta R13	No se renumeran en la U3
S	Servicio o contrato expuesto por el gateway	No se renumeran en la U3
F	Flujo de usuario	Base del inventario de flujos de la U3

Requisitos (R)

ID	Requisito (resumen del Caso 19)
R01	El cliente arma un pedido con los productos disponibles
R02	El pedido se prepara solo después de confirmar el pago
R03	El cliente puede anular una compra indicando el motivo
R04	Las ventas web se asocian a la caja de un cajero virtual (funcionario)
R05	Registro de clientes con sus datos y validación del correo
R06	El registro puede hacerlo el cliente en la web o el administrador en el local
R07	Administrar la disponibilidad (stock) de cada producto
R08	Boleta digital por cada venta, enviada al correo del cliente
R09	Reporte de ventas por período para el dueño
R10	Mantenedores de productos, usuarios y clientes
R11	Validar que el correo existe y está activo mediante una API de mensajería
R12	Autenticación de usuarios con acceso según el perfil
R13	Órdenes de despacho en el orden en que llegan

Flujos (F)

ID	Flujo	Rol	Requisitos
F01	Consultar catálogo	Anónimo	R01, R07
F02	Registrarse como cliente	Anónimo	R05, R06, R11
F03	Iniciar y cerrar sesión	Todos los roles	R12
F04	Comprar y pagar	CLIENTE	R01, R02, R04
F05	Ver mis pedidos	CLIENTE	R01, R03
F06	Anular un pedido	CLIENTE dueño, ADMIN	R03
F07	Administrar productos	ADMIN	R07, R10
F08	Atender la cola de despacho	DESPACHO, ADMIN	R02, R13
F09	Consultar reporte de ventas	DUENO, ADMIN	R09

2. Matriz de servicios

ID	R	F	Pantalla y acción	Propósito	Verbo y ruta	Acceso	Clasificación	Destino	Estado
S01	R01, R07	F01	Catálogo: cargar la lista (también llena la ficha y el carrito)	Listar productos con su disponibilidad	`GET /api/v1/productos`	Anónimo	Consulta	inventory	Implementado y conectado
S02	R01	F01	Ficha del producto	Ver el detalle de un producto	`GET /api/v1/productos/{id}`	Anónimo	Consulta	inventory	Implementado; el front no lo llama, la ficha usa los datos de S01
S03	R07, R10	F07	Panel: crear producto	Agregar un producto al catálogo	`POST /api/v1/productos`	ADMIN	Transaccional	inventory	Implementado; sin pantalla (el panel interno no está en la U2)
S04	R07, R10	F07	Panel: editar producto	Cambiar datos, precio o stock	`PUT /api/v1/productos/{id}`	ADMIN	Transaccional	inventory	Implementado; sin pantalla (el panel interno no está en la U2)
S05	R10	F07	Panel: eliminar producto	Quitar un producto del catálogo	`DELETE /api/v1/productos/{id}`	ADMIN	Transaccional	inventory	Implementado; sin pantalla (el panel interno no está en la U2)
S06	R12	F03	Login: botón "Ingresar"	Iniciar sesión	`POST /api/v1/auth/login`	Anónimo	Transaccional	auth (vía gateway)	Implementado y conectado
S07	R12	F03	Menú: "Cerrar sesión"	Cerrar sesión	`POST /api/v1/auth/logout`	Autenticado	Transaccional	auth (vía gateway)	Implementado y conectado
S08	R12	F03	Todas: cargar el usuario actual	Saber quién es y qué rol tiene	`GET /api/v1/auth/me`	Autenticado	Consulta	auth (vía gateway)	Implementado y conectado
S09	R05, R06, R11	F02	Registro: botón "Crear cuenta"	Registrar un cliente	`POST /api/v1/clientes`	Anónimo	Transaccional	clients	Implementado (S09V `POST /api/v1/clientes/{id}/verificar` para el código) (el registro sigue en localStorage)
S10	R05	F04, F05	Perfil propio	Ver el perfil propio	`GET /api/v1/clientes/me`	CLIENTE	Consulta	clients	Implementado
S11	R01, R04	F04	Checkout: botón "Continuar al pago"	Crear un pedido, asociado a la caja del cajero virtual	`POST /api/v1/pedidos`	CLIENTE	Transaccional	orders	Implementado y conectado
S12	R01, R08	F04	Pago y Boleta: cargar el pedido	Ver el detalle de un pedido	`GET /api/v1/pedidos/{id}`	CLIENTE dueño, DESPACHO (pedidos pagados), ADMIN	Consulta	orders	Implementado y conectado
S13	R03	F06	Mis pedidos: botón "Anular"	Anular un pedido con motivo	`POST /api/v1/pedidos/{id}/anular`	CLIENTE dueño, ADMIN	Transaccional	orders	Implementado y conectado

ID	R	F	Pantalla y acción	Propósito	Verbo y ruta	Acceso	Clasificación	Destino	Estado
S14	R02, R13	F08	Panel: cola de despacho	Listar pedidos pagados en orden de llegada	`GET /api/v1/pedidos?estado=PAGADO`	DESPACHO, ADMIN	Consulta	orders	Implementado; sin pantalla (el panel interno no está en la U2)
S15	R13	F08	Panel: cambiar estado	Marcar en despacho o entregado	`PATCH /api/v1/pedidos/{id}/estado`	DESPACHO, ADMIN	Transaccional	orders	Implementado; sin pantalla (el panel interno no está en la U2)
S16	R02, R08	F04	Pago: botón "Pagar"	Pagar un pedido propio y emitir la boleta	`POST /api/v1/pagos`	CLIENTE	Transaccional	payments	Implementado y conectado
S17	R02	F04	Sin pantalla (lo llama el proveedor)	Recibir el aviso de pago	`POST /api/v1/pagos/aviso`	Proveedor (secreto propio)	Integración	payments	Implementado (proveedor simulado)
S18	R01	F05	Mis pedidos: cargar la lista	Pedidos propios con datos actuales del producto	`POST /api/v1/graphql` (`misPedidos`, `pedido`)	Autenticado (las queries exigen CLIENTE)	Agregación	graphql-gateway	Implementado y conectado
S19	R09	F09	Panel: reporte de ventas	Ventas por período	`GET /api/v1/reportes/ventas`	DUENO, ADMIN	Analítica	orders	Implementado; sin pantalla (el panel interno no está en la U2)

Acceso en dos capas: el gateway revisa la sesión y el rol de la tabla (lista blanca en `gateway.py`). Para S12, S13 y S18 el gateway solo exige sesión, y la regla de propiedad la aplica orders (o graphql-gateway) con la identidad que le pasa el gateway.

3. Reglas comunes a todos los contratos

1. **Formato de error único:** `{ "error": { "code", "message", "request_id" } }`. S18 es la excepción cuando el error es de un campo: responde 200 con `errors[]` y `extensions.code` (formato GraphQL).
2. **Validación:** si faltan campos o tienen un tipo o valor inválido, la respuesta es **422** `VALIDATION_ERROR` e indica qué campos fallaron, nunca los valores recibidos.
3. **Correlación:** toda respuesta trae el header `X-Request-ID`. El gateway reutiliza el que manda el front si tiene entre 8 y 64 caracteres (letras, números, `-` o `_`); si no, genera uno. Lo propaga a todos los servicios internos.
4. **Lo calcula el servidor, nunca el navegador:** precio de cada línea, total del pedido, `cajero_id`, estado del pedido, monto y estado del pago, número de boleta, neto e IVA, y disponibilidad del producto (se calcula a partir del stock). Si el front manda esos campos, se ignoran.
5. **Identidad:** el gateway descarta los headers `X-Authenticated-*` que vengan del navegador y los vuelve a poner (`X-Authenticated-User`, `X-Authenticated-Roles`) con la sesión validada.
6. **Recurso ajeno:** si un cliente pide un pedido que no es suyo, la respuesta es la misma que si no existiera: 404 `ORDER_NOT_FOUND` en S12, S13 y S16, y `null` en la query `pedido` de S18.
7. **Idempotencia:** S11 y S16 exigen el header `Idempotency-Key` (8 a 64 caracteres: letras, números, `-` o `_`). Sin él, 400 `IDEMPOTENCY_KEY_REQUIRED`. Repetir la misma clave con el mismo contenido devuelve el mismo pedido o pago sin crear otro; repetirla con otro contenido da 409 `IDEMPOTENCY_CONFLICT`. La clave es por cliente.
8. **Servicio caído o lento:** 502 `UPSTREAM_UNAVAILABLE` si el servicio no responde y 504 `UPSTREAM_TIMEOUT` si se pasa del plazo (con `docker-compose.yml`: 5 s desde el gateway y 3 s desde orders, payments y graphql-gateway). En S18 una caída de inventory no tumba la respuesta: ver S18.
9. **Estados en mayúsculas**, igual que en el M2. Pedido: PENDIENTE_PAGO, PAGADO, EN_DESPACHO, ENTREGADO, ANULADO. Pago: PENDIENTE, APROBADO, RECHAZADO, REVERTIDO.
10. **Fechas** en ISO 8601 con zona UTC (por ejemplo `2026-11-06T14:16:29+00:00`).

11. Todos los datos de los ejemplos son ficticios y coinciden con `scripts/seed.py` y con las evidencias de `evidencias/u2/`.

4. Detalle de cada contrato

S01 `GET /api/v1/productos`

- **Entrada:** query opcional `categoria` (barras, bombones, trufas) y `pagina` (desde 1).
- **Salida:** 200 con la lista de productos, la página y el total.
- **Errores:** 422 si `categoria` no es una de las tres o `pagina` es menor que 1.

```
{
  "productos": [
    { "id": "650000000000000000000001", "nombre": "Chocolate Dubai", "descripcion": "Barra de chocolate c
on leche rellena de crema de pistacho", "categoria": "barras", "precio": 8990, "stock": 20, "disponible":
true, "imagen": "img/chocolate-dubai.jpg" },
    { "id": "650000000000000000000005", "nombre": "Trufas de chocolate", "descripcion": "Trufas artesanal
es con cobertura de chocolate", "categoria": "trufas", "precio": 5490, "stock": 0, "disponible": false,
"imagen": "img/trufas-de-chocolate.jpg" }
  ],
  "pagina": 1,
  "total": 5
}
```

S02 `GET /api/v1/productos/{id}`

- **Entrada:** `id` en la ruta.
- **Salida:** 200 con el producto.
- **Errores:** 400 `INVALID_ID` si el ID tiene formato inválido, 404 `PRODUCT_NOT_FOUND` si no existe.

```
{ "id": "650000000000000000000003", "nombre": "Bombones rellenos", "descripcion": "Bombones de chocolate
con centro de crema", "categoria": "bombones", "precio": 6990, "stock": 10, "disponible": true, "imagen":
"img/bombones-rellenos.jpg" }
```

S03 `POST /api/v1/productos`

- **Entrada:** nombre, descripcion (opcional), categoria (barras, bombones o trufas), precio (entero > 0), stock (entero ≥ 0) e imagen (opcional). `disponible` no se recibe: se calcula del stock.
- **Salida:** 201 con el producto creado.
- **Errores:** 401 sin sesión, 403 `FORBIDDEN` si no es ADMIN (lo revisan el gateway e inventory), 422 datos inválidos.

```
{ "nombre": "Trufas de café", "descripcion": "Trufas rellenas de ganache de café", "categoria": "trufas",
"precio": 5990, "stock": 12, "imagen": null }
```

S04 `PUT /api/v1/productos/{id}`

- **Entrada:** uno o más campos de S03 (solo se cambian los que vienen).
- **Salida:** 200 con el producto actualizado.
- **Errores:** 400 si no viene ningún campo o el ID es inválido, 401, 403, 404, 422.

```
{ "precio": 9490, "stock": 18 }
```

S05 `DELETE /api/v1/productos/{id}`

- **Entrada:** `id` en la ruta.
- **Salida:** 200 con un mensaje de confirmación.
- **Errores:** 400 ID inválido, 401, 403 si no es ADMIN, 404.

```
{ "mensaje": "Producto eliminado correctamente" }
```

S06 `POST /api/v1/auth/login`

- **Entrada:** correo y clave.
- **Salida:** 200 `{ "status": "OK" }` y la sesión queda en una cookie `httponly` (`session_token`) que expira según `AUTH_TOKEN_TTL`. El token no se devuelve en el cuerpo.
- **Errores:** 400 si el cuerpo no es JSON, 422 si falta correo o clave, 401 `INVALID_CREDENTIALS` (mismo mensaje si el correo no existe o la clave está mal). El límite de intentos (429) queda para la Etapa 4.

```
{ "correo": "cliente@correo.cl", "clave": "Cliente2026" }
```

```
{ "status": "OK" }
```

S07 `POST /api/v1/auth/logout`

- **Entrada:** cookie de sesión.
- **Salida:** 200 `{ "status": "OK" }`. La sesión se borra en auth y se elimina la cookie. Si no había sesión, responde igual.

S08 `GET /api/v1/auth/me`

- **Entrada:** cookie de sesión.
- **Salida:** 200 con el usuario actual.
- **Errores:** 401 `UNAUTHENTICATED` sin cookie, 401 `SESSION_INVALID` si la sesión expiró o no existe.

```
{ "user_id": "USR-0001", "roles": ["CLIENTE"], "nombre": "Valentina Soto" }
```

S09 `POST /api/v1/clientes`

Registro del cliente (`registro.html`). clients guarda el perfil y le pide a auth crear la credencial (rol CLIENTE, inactiva). La clave viaja una sola vez y auth guarda solo su hash bcrypt.

- **Entrada:** run, nombre, direccion, comuna, provincia, region, nacimiento (fecha), sexo, correo, telefono y clave (mínimo 6). El servidor vuelve a validar todo: RUN con dígito verificador, correo, teléfono y fecha no futura.
- **Salida:** 201 con `{id, verificado: false, codigo_simulado}`. `codigo_simulado` reemplaza al correo mientras no exista la mensajería (brecha R11).
- **Errores:** 409 `CLIENT_EXISTS` (RUN o correo ya registrado, mismo mensaje para no revelar cuál), 422 datos inválidos, 429 `RATE_LIMITED`, 502 si auth no responde.

```
{ "run": "12.345.678-5", "nombre": "Valentina Soto", "direccion": "Av. Providencia 1234", "comuna": "Prov
idencia", "provincia": "Santiago", "region": "Región Metropolitana de Santiago", "nacimiento": "1999-04-1
2", "sexo": "Femenino", "correo": "nueva@correo.cl", "telefono": "+56912345678", "clave": "ClaveFicticia
1" }
```

```
{ "id": "USR-3FA2B91C", "verificado": false, "codigo_simulado": "482913" }
```

S09V `POST /api/v1/clientes/{id}/verificar`

Verificación del correo (`verificar.html`). Si el código coincide con su hash, clients marca el perfil y auth activa la credencial. Antes de esto el login responde 401.

- **Entrada:** `{ "codigo": "482913" }` (6 dígitos).
- **Salida:** 200 `{ "id": "USR-3FA2B91C", "verificado": true }`.

- **Errores:** 400 `INVALID_CODE` (código malo, cuenta inexistente o ya verificada: mismo mensaje), 422, 429.

S10 `GET /api/v1/clientes/me`

Perfil propio. El id sale de la identidad que pone el gateway, nunca de la URL, así que no hay forma de pedir el perfil de otro.

- **Salida:** 200 con `{id, run (enmascarado), nombre, correo, telefono, direccion, comuna, provincia, region, verificado}`. No devuelve clave, código, fecha de nacimiento ni sexo (no los necesita ninguna pantalla).
- **Errores:** 401 sin sesión, 403 si el rol no es CLIENTE, 404 si el usuario no tiene perfil.

```
{ "id": "USR-0001", "run": "**.***.111-1", "nombre": "Valentina Soto", "correo": "cliente@correo.cl", "telefono": "+56911111111", "direccion": "Calle Ficticia 1", "comuna": "Providencia", "provincia": "Santiago", "region": "Region Metropolitana de Santiago", "verificado": true }
```

S11 `POST /api/v1/pedidos`

- **Entrada:** header `Idempotency-Key` y cuerpo con `lineas` (1 a 20, cada una con `producto_id` y `cantidad` de 1 a 20, sin repetir producto) y `despacho` (`direccion` de 3 a 120 caracteres, `comuna` de 2 a 60 y `telefono` opcional de hasta 20). Si viene precio, total, estado o cajero, se ignoran.
- **Salida:** 201 con el pedido en estado PENDIENTE_PAGO. orders reserva el stock en inventory (que devuelve el precio vigente), calcula subtotales y total, y **asigna el `cajero_id` del cajero virtual** USR-0005 (R04). Si se repite el mismo `Idempotency-Key` con el mismo cuerpo, devuelve el mismo pedido sin descontar stock otra vez.
- **Errores:** 400 `IDEMPOTENCY_KEY_REQUIRED`, 400 `BAD_REQUEST` si un producto aparece dos veces, 400 `INVALID_ID` si un `producto_id` tiene formato inválido, 401, 403 `FORBIDDEN` si no es CLIENTE, 404 `PRODUCT_NOT_FOUND`, 409 `STOCK_INSUFICIENTE`, 409 `IDEMPOTENCY_CONFLICT`, 422, 502 o 504 si inventory no responde.
- **Evidencia:** `evidencias/u2/pedido-idempotente/`.

```
{ "lineas": [ { "producto_id": "650000000000000000000001", "cantidad": 2 } ], "despacho": { "direccion": "Calle Ficticia 1", "comuna": "Providencia", "telefono": null } }
```

```
{
  "id": "PED-0003", "cliente_id": "USR-0001", "estado": "PENDIENTE_PAGO", "cajero_id": "USR-0005",
  "lineas": [ { "producto_id": "650000000000000000000001", "nombre": "Chocolate Dubai", "precio_unitario": 8990, "cantidad": 2, "subtotal": 17980 } ],
  "total": 17980,
  "creado_en": "2026-11-06T14:16:29+00:00",
  "historial": [ { "estado": "PENDIENTE_PAGO", "fecha": "2026-11-06T14:16:29+00:00", "por": "USR-0001" } ],
  "despacho": { "direccion": "Calle Ficticia 1", "comuna": "Providencia", "telefono": null }
}
```

S12 `GET /api/v1/pedidos/{id}`

- **Entrada:** `id` en la ruta.
- **Salida:** 200 con el pedido, en el mismo formato de S11. Si está pagado trae `boleta` (R08 parcial); si está anulado trae `motivo_anulacion`. El `despacho` solo lo ven el dueño, DESPACHO y ADMIN (M7). Lo usan `pago.html` y `boleta.html`.
- **Errores:** 400 `INVALID_ID`, 401, 404 `ORDER_NOT_FOUND` si no existe o es de otro cliente (DESPACHO tampoco ve pedidos sin pagar).

```
{
  "id": "PED-0003", "cliente_id": "USR-0001", "estado": "PAGADO", "cajero_id": "USR-0005",
  "lineas": [ { "producto_id": "650000000000000000000001", "nombre": "Chocolate Dubai", "precio_unitario": 8990, "cantidad": 2, "subtotal": 17980 } ],
```

```
  "total": 17980,
  "creado_en": "2026-11-06T14:16:29+00:00",
  "historial": [
    { "estado": "PENDIENTE_PAGO", "fecha": "2026-11-06T14:16:29+00:00", "por": "USR-0001" },
    { "estado": "PAGADO", "fecha": "2026-11-06T14:16:31+00:00", "por": "payments" }
  ],
  "despacho": { "direccion": "Calle Ficticia 1", "comuna": "Providencia", "telefono": null },
  "boleta": { "numero": "B-000002", "fecha": "2026-11-06T14:16:31+00:00", "neto": 15109, "iva": 2871, "to
tal": 17980 }
}
```

S13 `POST /api/v1/pedidos/{id}/anular`

- **Entrada:** `motivo` (obligatorio, de 3 a 200 caracteres, RF3).
- **Salida:** 200 con el pedido en estado ANULADO y `motivo_anulacion`. El registro no se borra. El cambio de estado es atómico: si llegan dos anulaciones juntas, solo una devuelve el stock. Si inventory no responde al devolver, el pedido igual queda anulado, se guarda `stock_devuelto: false` y se registra el evento `stock_return_failed` en el log.
- **Errores:** 400 `INVALID_ID`, 401, 404 `ORDER_NOT_FOUND` ajeno o inexistente, 409 `TRANSICION_INVALIDA` si el pedido ya no está en PENDIENTE_PAGO, 422 sin motivo o muy corto.

```
{ "motivo": "me equivoque de producto" }
```

S14 `GET /api/v1/pedidos?estado=PAGADO`

- **Entrada:** query opcional `estado` (en mayúsculas), `pagina` (desde 1) y `por_pagina` (1 a 50, por defecto 20).
- **Salida:** 200 con `{ pedidos, pagina, total }`, del más antiguo al más nuevo según `creado_en` (RF13). Sin `estado`, DESPACHO recibe PAGADO, EN_DESPACHO y ENTREGADO, y ADMIN recibe todos.
- **Errores:** 401, 403 si el rol no es DESPACHO ni ADMIN (lo revisa el gateway) o si DESPACHO pide un estado sin pagar (PENDIENTE_PAGO o ANULADO), 422 estado desconocido.

```
{
  "pedidos": [
    { "id": "PED-0001", "cliente_id": "USR-0001", "estado": "PAGADO", "cajero_id": "USR-0005",
      "lineas": [ { "producto_id": "650000000000000000000001", "nombre": "Chocolate Dubai", "precio_unita
rio": 8990, "cantidad": 1, "subtotal": 8990 },
                  { "producto_id": "650000000000000000000003", "nombre": "Bombones rellenos", "precio_uni
tario": 6990, "cantidad": 2, "subtotal": 13980 } ],
      "total": 22970, "creado_en": "2026-11-02T15:00:00+00:00",
      "despacho": { "direccion": "Calle Ficticia 123", "comuna": "Providencia", "telefono": "+5690000000
0" },
      "boleta": { "numero": "B-000001", "fecha": "2026-11-02T15:00:00+00:00", "neto": 19303, "iva": 3667,
"total": 22970 } }
  ],
  "pagina": 1,
  "total": 1
}
```

(Ejemplo resumido: cada pedido trae también su `historial`.)

S15 `PATCH /api/v1/pedidos/{id}/estado`

- **Entrada:** `estado` nuevo. Solo se permite PAGADO → EN_DESPACHO y EN_DESPACHO → ENTREGADO. PAGADO lo pone únicamente payments.
- **Salida:** 200 con el pedido actualizado y una entrada nueva en `historial`.
- **Errores:** 400 `INVALID_ID`, 401, 403, 404 `ORDER_NOT_FOUND`, 409 `TRANSICION_INVALIDA` (por ejemplo, de PENDIENTE_PAGO a ENTREGADO, o pedir PAGADO o ANULADO), 422 estado desconocido.

```
{ "estado": "EN_DESPACHO" }
```

S16 `POST /api/v1/pagos`

- **Entrada:** header `Idempotency-Key` y `pedido_id`. El monto lo saca payments desde orders; no se reciben datos de tarjeta (M7).
- **Salida:** 201 con el pago. Con el proveedor simulado en modo `inmediato` (el de `docker-compose.yml`), el pago vuelve APROBADO y payments le avisa a orders, que pasa el pedido a PAGADO y numera la boleta. En modo `aviso`, el pago queda PENDIENTE hasta que llega S17. Si el pago está aprobado pero orders no alcanzó a registrarlo, viene `pedido_sincronizado: false` y payments reintenta cada 15 s.
- **Errores:** 400 `IDEMPOTENCY_KEY_REQUIRED`, 401, 403 si no es CLIENTE, 404 `ORDER_NOT_FOUND` pedido ajeno o inexistente, 409 `PEDIDO_NO_PAGABLE` si el pedido no está en PENDIENTE_PAGO, 409 `IDEMPOTENCY_CONFLICT`, 422, 502 o 504 si orders no responde al consultar el pedido.
- **Nota:** el proveedor es un doble de prueba que respeta el contrato (no es Webpay).
- **Evidencia:** `evidencias/u2/pago-idempotente/` y `evidencias/u2/pago-orders-caido/`.

```
{ "pedido_id": "PED-0003" }
```

```
{
  "id": "PAG-0002", "pedido_id": "PED-0003", "monto": 17980, "estado": "APROBADO", "proveedor": "simulado",
  "creado_en": "2026-11-06T14:16:31+00:00",
  "pedido_sincronizado": true,
  "boleta": { "numero": "B-000002", "fecha": "2026-11-06T14:16:31+00:00", "neto": 15109, "iva": 2871, "total": 17980 }
}
```

S17 `POST /api/v1/pagos/aviso`

- **Entrada:** header `X-Proveedor-Secret` (secreto propio del proveedor) y cuerpo con `pago_id`, `resultado` (APROBADO o RECHAZADO) y `referencia_proveedor`. El gateway la deja pasar sin sesión porque la llama el proveedor; payments revisa el secreto.
- **Salida:** 200 con el pago. Si el resultado es APROBADO, payments avisa a orders y el pedido pasa a PAGADO con su boleta. Solo payments decide que un pedido está pagado. Un aviso repetido no cambia nada (si el pago estaba aprobado pero sin sincronizar, aprovecha de reintentar). Si el pedido ya estaba pagado con otro pago, este queda REVERTIDO.
- **Errores:** 401 `INVALID_PROVIDER_SIGNATURE` secreto inválido, 404 `PAYMENT_NOT_FOUND`, 422.

```
{ "pago_id": "PAG-0002", "resultado": "APROBADO", "referencia_proveedor": "SIM-PAG-0002" }
```

S18 `POST /api/v1/graphql`

- **Entrada:** cuerpo `{ query, variables }`. Solo POST y sin GraphiQL. Dos queries, ambas solo para CLIENTE:
  - `misPedidos(pagina, porPagina)`: pedidos propios del más antiguo al más nuevo (`porPagina` de 1 a 20, por defecto 10). Devuelve `{ items, pagina, porPagina, total }`.
  - `pedido(id)`: un pedido propio; si es ajeno o no existe, `null`.
- **Tipos:** Pedido (`id`, `estado`, `total`, `creadoEn`, `boleta`, `despacho`, `lineas`), Linea (`productoId`, `nombre`, `cantidad`, `precioUnitario`, `subtotal`, `producto`, `pedido`), Producto (`id`, `nombre`, `categoria`, `imagen`, `disponible`, `stock`), Boleta y Despacho (`direccion` y `comuna`, sin teléfono).
- **Salida:** 200 con `data`. Junta orders (pedidos) e inventory (estado actual de cada producto) con una sola llamada a inventory por consulta (DataLoader).
- **Límites:** profundidad 6, 10 alias y 1000 tokens. Si se pasan, la consulta se rechaza entera.
- **Errores:** 401 sin sesión (HTTP, lo responde el gateway). Los demás van en `errors[]` con `extensions.code`: `FORBIDDEN` si el rol no es CLIENTE, `BAD_REQUEST` paginación fuera de rango, `FORBIDDEN_FIELD` si se pide `stock` sin ser ADMIN, `UPSTREAM_UNAVAILABLE`, `UPSTREAM_TIMEOUT` o `UPSTREAM_ERROR` si un servicio falla. Si inventory no responde, los pedidos igual llegan y `producto` viene `null` con su error (error parcial).

- **Evidencia:** `evidencias/u2/graphql-agregacion/` y `evidencias/u2/graphql-error-parcial/`.

```
{ "query": "query MisPedidos($pagina: Int!) { misPedidos(pagina: $pagina, porPagina: 10) { total items { id estado total boleta { numero } lineas { nombre cantidad producto { disponible imagen } } } } }", "variables": { "pagina": 1 } }
```

```
{ "data": { "misPedidos": { "total": 2, "items": [
  { "id": "PED-0001", "estado": "PAGADO", "total": 22970, "boleta": { "numero": "B-000001" },
    "lineas": [ { "nombre": "Chocolate Dubai", "cantidad": 1, "producto": { "disponible": true, "imagen": "img/chocolate-dubai.jpg" } },
                { "nombre": "Bombones rellenos", "cantidad": 2, "producto": { "disponible": true, "imagen": "img/bombones-rellenos.jpg" } } ] },
  { "id": "PED-0003", "estado": "PENDIENTE_PAGO", "total": 15980, "boleta": null,
    "lineas": [ { "nombre": "Chocolate Dubai", "cantidad": 1, "producto": { "disponible": true, "imagen": "img/chocolate-dubai.jpg" } },
                { "nombre": "Bombones rellenos", "cantidad": 1, "producto": { "disponible": true, "imagen": "img/bombones-rellenos.jpg" } } ] }
] } } }
```

Con inventory caído (error parcial, resumido):

```
{ "data": { "misPedidos": { "items": [ { "id": "PED-0001", "estado": "PAGADO", "lineas": [ { "nombre": "Chocolate Dubai", "producto": null } ] } ] } },
  "errors": [ { "message": "inventory no esta disponible", "path": ["misPedidos", "items", 0, "lineas", 0, "producto"], "extensions": { "code": "UPSTREAM_UNAVAILABLE" } } ] }
```

S19 `GET /api/v1/reportes/ventas`

- **Entrada:** query opcional `desde` y `hasta` (fechas `AAAA-MM-DD`). Sin fechas, cuenta todo.
- **Salida:** 200 con la cantidad de pedidos, el total vendido y el detalle por producto (de mayor a menor monto). Solo cuenta pedidos PAGADO, EN_DESPACHO o ENTREGADO, y los ubica en el día en que se pagaron (`pagado_en`), no en el que se crearon. No incluye datos personales de clientes.
- **Errores:** 400 `BAD_REQUEST` si `desde` es mayor que `hasta`, 401, 403 si el rol no es DUENO ni ADMIN, 422 fechas inválidas.

```
{
  "desde": "2026-11-01", "hasta": "2026-11-30",
  "cantidad_pedidos": 1, "total_ventas": 22970,
  "por_producto": [
    { "producto_id": "650000000000000000000003", "nombre": "Bombones rellenos", "cantidad": 2, "monto": 13980 },
    { "producto_id": "650000000000000000000001", "nombre": "Chocolate Dubai", "cantidad": 1, "monto": 8990 }
  ]
}
```

(Datos de `scripts/seed.py` recién reseteado.)

5. Estado de los requisitos

Requisito	Estado	Dónde se cubre o se declara
R01	**Cubierto**	S01, S11, S12, S18
R02	**Cubierto.** El pedido pasa a PAGADO solo cuando payments lo confirma, y solo los pagados entran a la cola de despacho	S14, S16, S17
R03	**Cubierto.** Anulación con motivo, sin borrar el registro y con devolución de stock	S13
R04	**Cubierto.** orders asigna el `cajero_id` USR-0005 (cajero virtual) a cada pedido web	S11

Requisito	Estado	Dónde se cubre o se declara
R05	Registro con RUN validado y verificación del correo (simulada, sin mensajería) en el servicio clients	S09, S09V, S10
R06	**Brecha declarada.** No hay registro por el administrador	Informe
R07	**Cubierto** en el backend; sin pantalla (el panel interno no está en la U2)	S01, S03, S04
R08	**Parcial.** Se emite una boleta numerada con neto e IVA (19 %) al pagar y se ve en `boleta.html`; no se envía por correo	S12, S16, informe
R09	**Cubierto** en el backend; sin pantalla (el panel interno no está en la U2)	S19
R10	**Parcial.** Mantenedor de productos sí (S03 a S05); usuarios y clientes no	Informe
R11 (y la validación de correo de R05)	**Brecha declarada.** La validación se simula en `verificar.html`, sin envío real	M7, informe
R12	**Cubierto.** Sesión con cookie httponly y permisos por rol en el gateway y en cada servicio	S06, S07, S08, M4
R13	**Cubierto** en el backend; sin pantalla (el panel interno no está en la U2)	S14, S15

6. Cambios respecto de la versión anterior (INCR-04)

Servicio	Antes	Ahora (código de `main`)
S02	Lo usaba la ficha del producto	El front no lo llama: la ficha usa los datos de S01
S09, S09V, S10	Confirmados con `backend/clients/main.py` y `tests/api/test_clientes.py`	Implementados
S11	`direccion_despacho` como texto; respuesta sin `cliente_id`, `creado_en` ni `historial`	`despacho { direccion, comuna, telefono }`; respuesta completa; errores 400 `IDEMPOTENCY_KEY_REQUIRED`, 400 producto repetido, 404 `PRODUCT_NOT_FOUND`, 409 `STOCK_INSUFICIENTE` y 409 `IDEMPOTENCY_CONFLICT`
S12	Flujo F05, solo dueño y ADMIN; respuesta resumida con `fecha`	Flujo F04 (lo usan Pago y Boleta; Mis pedidos usa S18); también DESPACHO (pedidos pagados); trae `boleta` y `historial`
S14	Respuesta `{ pedidos }` con `fecha_pago`	`{ pedidos, pagina, total }` con el pedido completo, paginado y ordenado por `creado_en`
S16	Requisitos R02 y R04; el pago volvía PENDIENTE	Requisitos R02 y R08 (emite la boleta); vuelve APROBADO con la boleta (modo inmediato) o PENDIENTE (modo aviso); se agrega `pedido_sincronizado` y el error 409 `PEDIDO_NO_PAGABLE`
S17	Sin códigos de error	401 `INVALID_PROVIDER_SIGNATURE`, 404 `PAYMENT_NOT_FOUND` y estado REVERTIDO
S18	`misPedidos` devolvía una lista	`misPedidos` devuelve `{ items, pagina, porPagina, total }`; se agrega `pedido(id)`, los límites y los códigos de error
S19	Contaba por fecha del pedido	Cuenta por `pagado_en`; agrega `producto_id` en el detalle
R08	Brecha	Parcial (boleta sin correo)

M2. Modelo del dominio

Diagrama: `docs/uml-dominio.drawio` (se abre en app.diagrams.net)

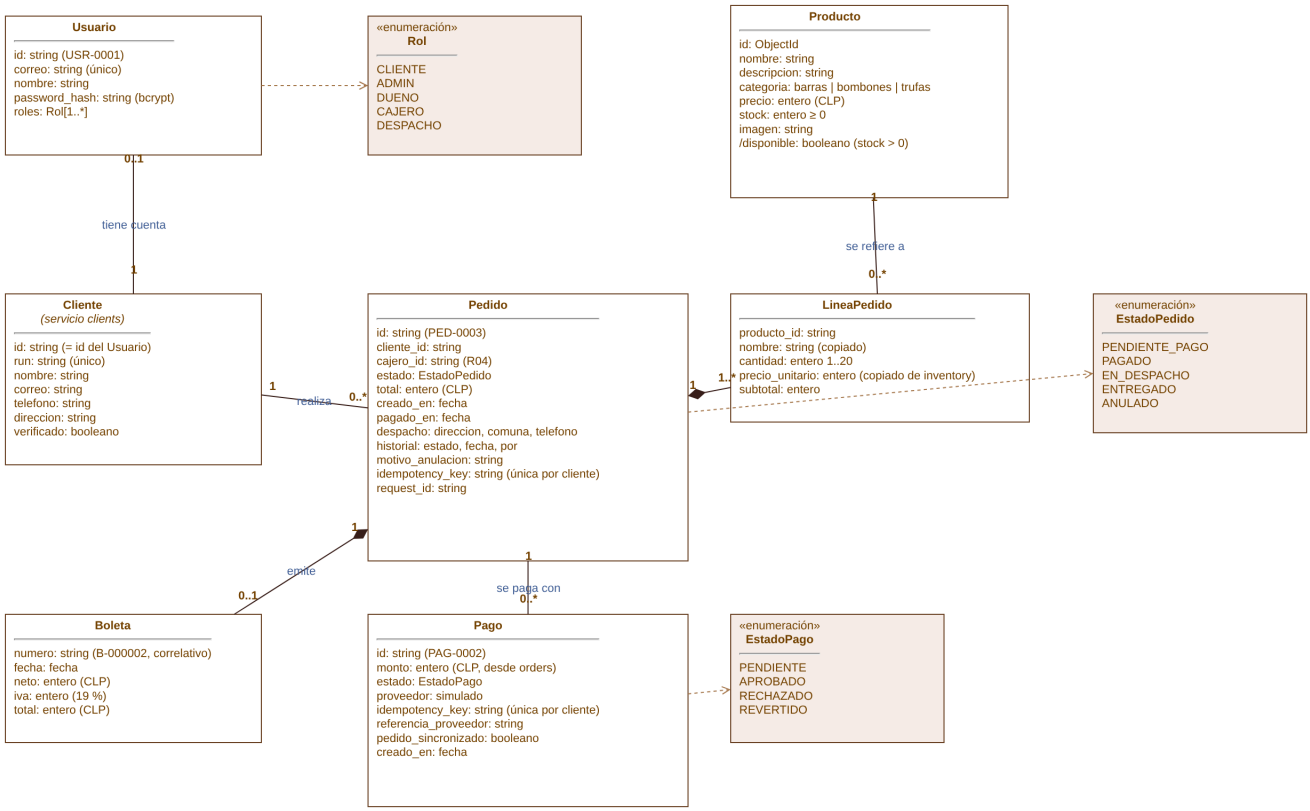


Diagrama UML del dominio y servicios públicos (fuente editable: docs/uml-dominio.drawio)

Estado: confirmado con el código de `main` (bdd26ff): `scripts/seed.py`, `inventory/models.py`, `orders/main.py` y `payments/main.py`. Las reglas usan el prefijo **D** (dominio) para no confundirlas con los requisitos R del M1.

1. Clases

Clase	Qué representa	Servicio dueño
Usuario	Cuenta para iniciar sesión, con un rol	auth
Rol	CLIENTE, ADMIN, DUENO, CAJERO, DESPACHO	auth
Cliente	Persona que compra, identificada por su RUN (perfil: RUN, nombre, contacto, dirección, verificado). Su id es el mismo del Usuario con rol CLIENTE que crea auth al registrarse	clients
Producto	Producto del catálogo (categoría barras, bombones o trufas), con precio y stock. `disponible` se calcula desde el stock	inventory
Pedido	Compra de un cliente, con su estado, el cajero virtual asignado, la dirección de despacho y el historial de estados	orders
LineaPedido	Un producto dentro de un pedido, con cantidad y precio	orders
Boleta	Comprobante de un pedido pagado, con número correlativo, neto e IVA. Va dentro del pedido	orders
Pago	Intento de pago de un pedido (PENDIENTE, APROBADO, RECHAZADO o REVERTIDO)	payments

2. Multiplicidades

Relación	Multiplicidad	Lectura
Cliente – Usuario	1 a 0..1	Un cliente puede tener o no una cuenta. Un cliente registrado por el admin o atendido en caja puede no tenerla
Cliente – Pedido	1 a 0..*	Un cliente puede tener cero o muchos pedidos. Cada pedido es de un solo cliente
Pedido – LineaPedido	1 a 1..* (composición)	Un pedido tiene al menos una línea. Si el pedido no existe, sus líneas tampoco
LineaPedido – Producto	0..* a 1	Cada línea apunta a un producto. Un producto puede estar en muchas líneas
Pedido – Pago	1 a 0..*	Un pedido puede tener varios intentos de pago (por ejemplo, uno rechazado y otro aprobado)
Pedido – Boleta	1 a 0..1 (composición)	Un pedido tiene boleta solo cuando está pagado

3. Reglas del dominio

ID	Regla	Dónde se aplica
D01	El precio de cada línea se copia desde inventory al crear el pedido. No se acepta el precio que manda el front	orders
D02	El stock nunca queda negativo. Si no alcanza, el pedido se rechaza	inventory
D03	Un `Idempotency-Key` repetido devuelve el mismo pedido o pago, sin crear otro	orders, payments
D04	Un cliente solo puede ver y anular sus propios pedidos (uno ajeno responde 404). DESPACHO ve los pagados y ADMIN todos	orders
D05	Al anular un pedido, el stock de sus líneas se devuelve a inventory	orders, inventory
D06	El total del pedido es la suma de los subtotales de sus líneas	orders
D07	Un pedido tiene como máximo un pago APROBADO. Si llega un segundo aprobado, queda REVERTIDO	payments, orders
D08	La boleta se emite solo cuando el pedido pasa a PAGADO	orders
D09	El correo del usuario es único. El RUN del cliente es único y se valida con dígito verificador en clients (además del front)	auth, clients
D10	La contraseña se guarda solo como hash bcrypt	auth
D11	Solo DESPACHO o ADMIN cambian un pedido a EN_DESPACHO o ENTREGADO	orders
D12	Cada pedido web queda asociado al cajero virtual USR-0005	orders
D13	El pedido guarda una copia de los nombres, precios y dirección: si cambian después, el pedido no cambia	orders

4. Estados del pedido

Desde	Hacia	Quién	Cuándo
(nuevo)	PENDIENTE_PAGO	CLIENTE	Al crear el pedido
PENDIENTE_PAGO	PAGADO	payments	Llega el aviso de pago aprobado
PENDIENTE_PAGO	ANULADO	CLIENTE dueño, ADMIN	Anulación (devuelve stock)
PAGADO	EN_DESPACHO	DESPACHO, ADMIN	Sale a reparto
EN_DESPACHO	ENTREGADO	DESPACHO, ADMIN	Se entrega

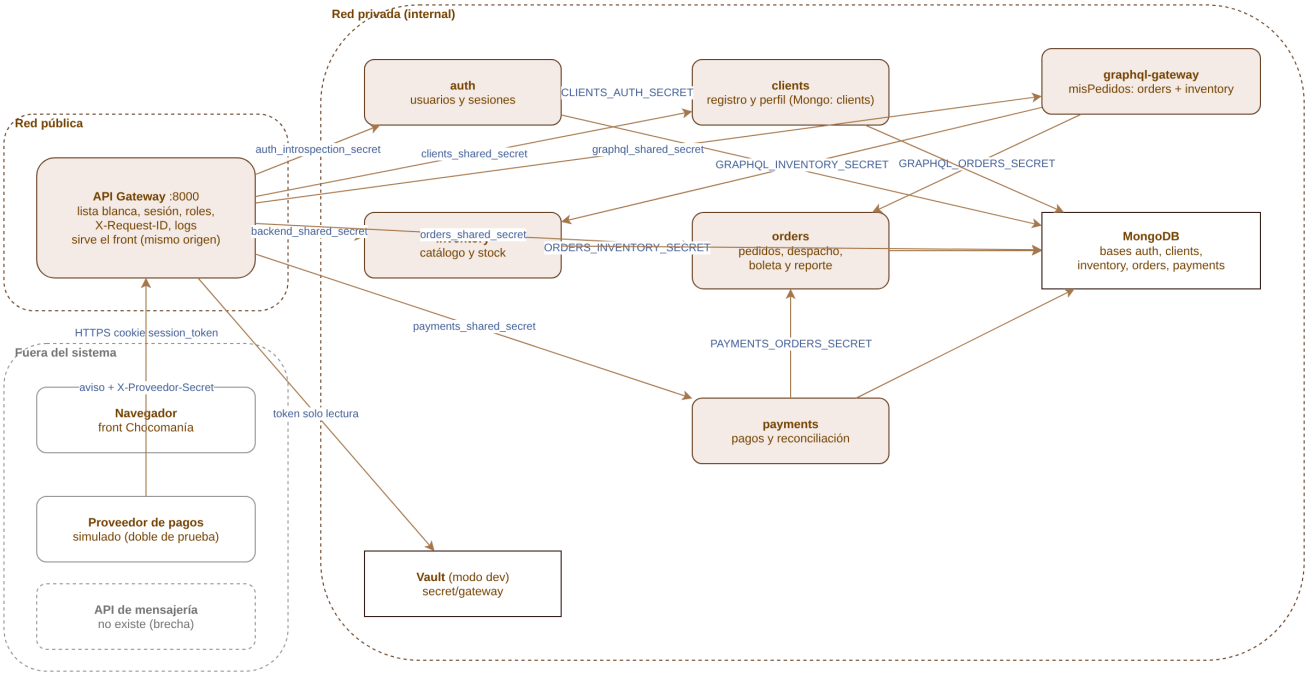
5. Confirmación con el código

Los estados y atributos se confirmaron con orders y payments (PR #32 y #33). Cambios respecto de la primera versión del diagrama:

- Los ID de usuario, pedido y pago son textos con prefijo (`USR-0001`, `PED-0003`, `PAG-0002`); los productos usan ObjectId.
- Usuario tiene `roles` (lista) y `nombre`; Producto usa `categoria` (barras, bombones, trufas) e `imagen`.
- Pedido agrega `cliente_id`, `cajero_id`, `creado_en`, `pagado_en`, `despacho`, `historial` y `motivo_anulacion`.
- Boleta usa número con formato `B-000002`, neto e IVA, y no tiene correo de destino (R08 parcial).
- Pago agrega el estado REVERTIDO, el proveedor y `pedido_sincronizado`.
- Cliente está implementado en el servicio clients; comparte el id con su Usuario (1 a 0..1).

M3. Matriz de sistemas, mapa de componentes y secuencia de una compra

Mapa de componentes: `docs/componentes.drawio` (se abre en app.diagrams.net).



Mapa de componentes y despliegue (fuente editable: docs/componentes.drawio)

Fuente: código de `main` (commit `6907414`, con orders, payments, GraphQL y el front ya mergeados), `docker-compose.yml` y las "Notas para el informe" de los PR #32 (orders), #33 (payments) y #37 (GraphQL).

1. Sistemas

Sistema	Responsabilidad	Datos que maneja (base propia)	Servicios del M1	Depende de	Contrato	Credencial	Respuesta ante fallos	Tipo de dependencia
gateway	Único punto de entrada: lista blanca, sesión, roles, `X-Request-ID` y reenvío. También sirve el front (archivos estáticos) en el mismo origen, `http://localhost:8000`	No guarda datos	Todos (S01 a S19)	auth, inventory, orders, payments, graphql-gateway, Vault	`docs/m1-servicios.md` y `docs/m6-rutas-secretos.md`	Token de Vault de solo lectura; un secreto por servicio	Plazo de 5 s: 504 si un servicio no responde a tiempo, 502 si no está disponible; 503 si Vault no responde y no hay caché	Real
auth	Usuarios, contraseñas (bcrypt) y sesiones	Base `auth`: usuarios y sesiones (solo el hash del token)	S06, S07, S08	MongoDB	`/login`, `/introspect`, `/logout`	`auth_introspection_secret`	Si auth cae, el gateway no puede validar sesiones: las rutas con sesión fallan y el catálogo anónimo sigue funcionando	Real

Sistema	Responsabilidad	Datos que maneja (base propia)	Servicios del M1	Depende de	Contrato	Credencial	Respuesta ante fallos	Tipo de dependencia
inventory	Catálogo, precios y stock	Base `inventory`: productos	S01 a S05 y operaciones internas de stock y de lectura por lote	MongoDB	`/productos`, `/interno/reservas`, `/interno/devoluciones`, `/interno/productos/lote`	`backend_shared_secret` (gateway), `ORDERS_INVENTORY_SECRET` (orders) y `GRAPHQL_INVENTORY_SECRET` (graphql-gateway, solo lectura)	Descuenta cada línea de forma atómica, solo si alcanza el stock; si una línea falla, devuelve las ya descontadas y responde 409 `STOCK_INSUFICIENTE`	Real
orders	Pedidos, estados, anulación, despacho, boleta y reporte de ventas	Base `orders`: pedidos (con líneas, historial, despacho, `cajero_id`, `request_id` y boleta) y contador de boletas	S11 a S15 y S19	inventory, MongoDB	`/pedidos`, `/reportes/ventas`, `/interno/pedidos/*`	`orders_shared_secret` (gateway), `PAYMENTS_ORDERS_SECRET` (payments) y `GRAPHQL_ORDERS_SECRET` (graphql-gateway, solo para leer pedidos)	Plazo de 3 s hacia inventory (menor que el del gateway, para saber qué servicio falló): 504 si tarda, 502 si está caído. Si la devolución de stock falla al anular, el pedido queda ANULADO con `stock_devuelto: false` y el evento `stock_return_failed` en el log	Real
payments	Pagos, idempotencia, aviso del proveedor y reconciliación con orders	Base `payments`: pagos (con `idempotency_key`, `request_id` y `pedido_sincronizado`)	S16, S17	orders, proveedor de pagos, MongoDB	`/pagos`, `/pagos/aviso`	`payments_shared_secret` (gateway), `PAYMENTS_ORDERS_SECRET` y `PAYMENT_PROVIDER_SECRET`	Si orders no responde al confirmar un pago, el pago queda APROBADO con `pedido_sincronizado: false` y se reintenta solo cada 15 s, con un aviso repetido o con un reintento del cliente	Real
graphql-gateway	Agregar en una consulta los pedidos del cliente (orders) con el estado actual de cada producto (inventory)	No guarda datos	S18 (`misPedidos` y `pedido`)	orders, inventory	`POST /graphql` (solo POST, sin GraphiQL)	`graphql_shared_secret` (gateway → graphql-gateway), `GRAPHQL_ORDERS_SECRET` y `GRAPHQL_INVENTORY_SECRET`	Plazo de 3 s hacia cada servicio. Si inventory no responde, devuelve los pedidos con `producto: null` y el detalle en `errors` (error parcial). Si orders no responde, devuelve el error `UPSTREAM_UNAVAILABLE` o `UPSTREAM_TIMEOUT` sin datos	Real
Proveedor de pagos	Aprobar o rechazar el cobro	Ninguno propio	S17 (llama al sistema)	—	Aviso `{pago_id, resultado, referencia_proveedor}`	`PAYMENT_PROVIDER_SECRET`	Modo `aviso`: el pago queda PENDIENTE hasta que llega el aviso. Un aviso repetido o tardío no cambia el resultado	**Doble de prueba** (simulador dentro de payments, no es Webpay)

Sistema	Responsabilidad	Datos que maneja (base propia)	Servicios del M1	Depende de	Contrato	Credencial	Respuesta ante fallos	Tipo de dependencia
clients	Registro de clientes con RUN y perfil, y verificación del correo	Base `clients`: clientes (RUN y correo únicos, dirección, teléfono, `verificado`, hash del código). No guarda claves	S09, S09V, S10	auth (crear y activar la credencial), MongoDB	`/clientes`, `/clientes/{id}/verificar`, `/clientes/me`	`clients_shared_secret` (gateway) y `CLIENTS_AUTH_SECRET` (clients → auth)	Plazo de 3 s hacia auth: 504 o 502 y no se crea el perfil. Si el perfil falla después de crear la credencial (registro duplicado en carrera), clients revierte la credencial inactiva en auth	Real
Vault	Guardar los secretos del gateway	`secret/gateway`	—	—	API de Vault	Token raíz (solo `vault-init`)	Ver `docs/m6-rutas-secretos.md`, sección 6	Real, en modo dev
MongoDB	Persistencia de cada servicio	Una base por servicio	—	—	Driver de Mongo	Usuario raíz (brecha del M6)	Si cae, todos los servicios con datos fallan	Real
API de mensajería	Validar correos y enviar boletas (R08, R11)	—	—	—	—	—	—	**No existe** en la U2: brecha declarada

2. Despacho y boletas

No son servicios aparte, sino partes de orders, como se acordó en los recortes:

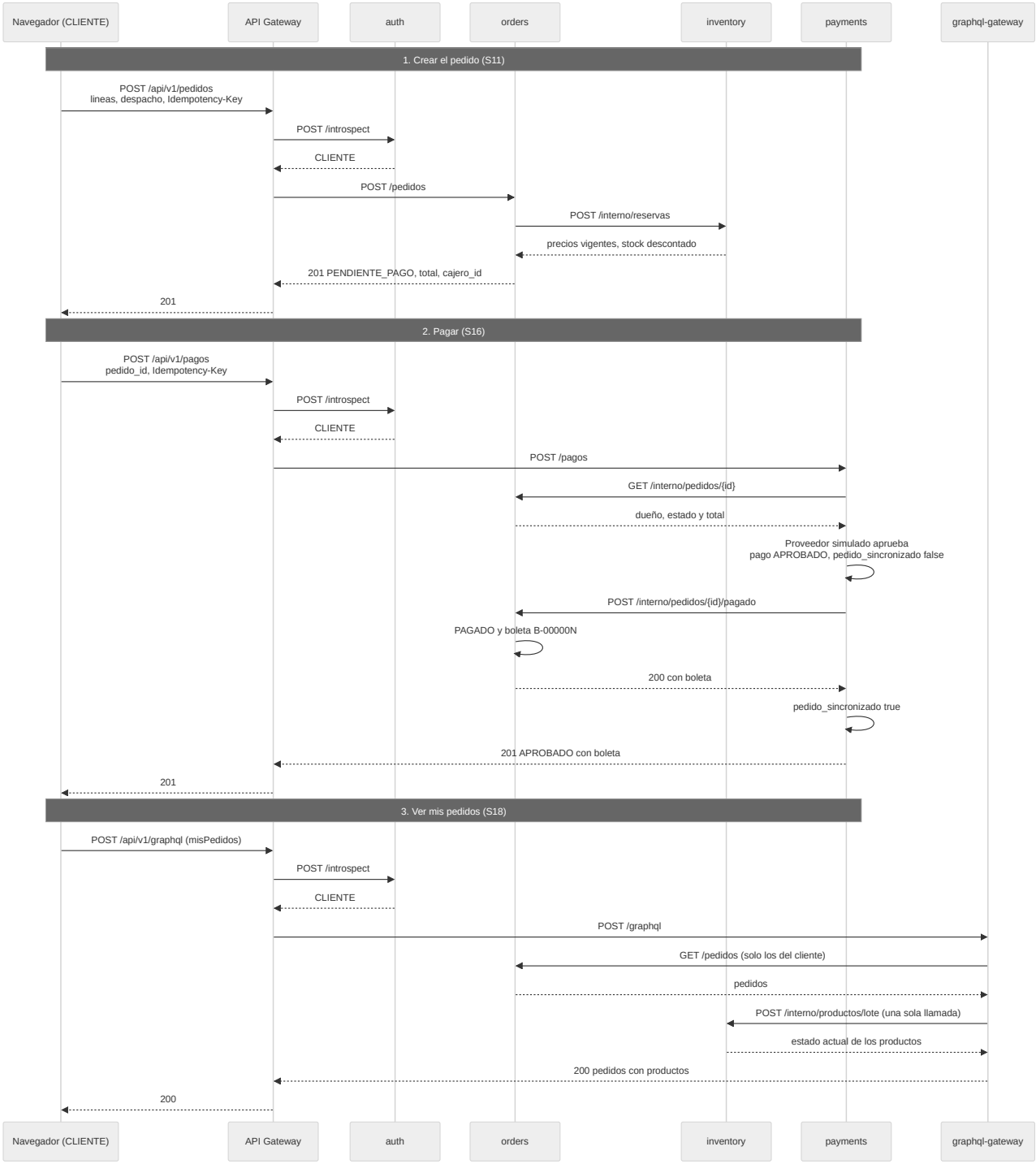
- **Despacho:** estados PAGADO → EN_DESPACHO → ENTREGADO, que solo mueven DESPACHO o ADMIN (S14 y S15). La cola muestra los pedidos vendidos del más antiguo al más nuevo según su creación (R13).
- **Reporte de ventas:** cuenta cada venta el día en que se pagó (`pagado_en`), no el día en que se creó el pedido.
- **Boleta (R08 parcial):** orders la numera (`B-000001`, `B-000002`…) solo cuando payments marca el pedido como PAGADO, con neto, IVA 19 % y total. El envío por correo es brecha declarada.

3. Reglas entre sistemas

1. **Solo payments decide que un pedido está pagado.** orders no tiene ninguna ruta pública para eso; payments usa `/interno/pedidos/{id}/pagado`, con su propio secreto.
2. **El servidor calcula los montos.** El precio de cada línea sale de inventory, el total lo calcula orders y el monto del pago lo saca payments desde orders. Si el front manda precio, total, monto o estado, se ignoran.
3. **Cada servicio solo modifica sus propios datos.** orders no toca el stock directamente: se lo pide a inventory.
4. **No se cobra dos veces:** `Idempotency-Key` obligatorio en pedidos y pagos, un segundo pago del mismo pedido responde 409 y, si dos pagos se aprobaran a la vez, orders acepta solo el primero.
5. **Anular no es borrar:** el pedido pasa a ANULADO con motivo e historial, y el stock se devuelve. Solo se anula un pedido PENDIENTE_PAGO; si no, 409 `TRANSICION_INVALIDA`.

4. Secuencia de una compra

Recorrido según el código de `main` (`6907414`), con el proveedor en modo inmediato (el que viene por defecto). Desde el navegador, el front hace estas mismas llamadas.



Si orders no responde en el último paso, el pago queda APROBADO con `pedido_sincronizado: false` y payments reintenta solo hasta que orders responde.

Evidencias de cada tramo

Tramo	Evidencia	`request_id`	Qué muestra
Crear pedido	`evidencias/u2/pedido-idempotente/`	`crear-pedido-0001`	Precio del front ignorado, reintento sin duplicar, el mismo ID en gateway, auth, inventory, orders y Mongo
Pagar	`evidencias/u2/pago-idempotente/`	`pagar-pedido-0001`	Monto del front ignorado, pago aprobado, pedido PAGADO con boleta B-000002, reintento sin duplicar y segundo pago rechazado

Tramo	Evidencia	request_id	Qué muestra
Pagar con orders caído	`evidencias/u2/pago-orders-caido/`	`aviso-orders-caido-1`	Pago APROBADO sin sincronizar mientras orders estaba apagado y sincronizado solo al volver
Ver mis pedidos	`evidencias/u2/graphql-agregacion/`	`gql-mispedidos-0002`	Pedidos de orders con productos de inventory, con una sola llamada a inventory
Ver mis pedidos con inventory caído	`evidencias/u2/graphql-error-parcial/`	`gql-inventory-caido-0002`	Pedidos visibles y `producto: null` con `UPSTREAM_UNAVAILABLE` en `errors`

Son ejecuciones distintas: el seed reutiliza los ID `PED-0003` y `PAG-0002` en cada reset. La evidencia `pago-idempotente` es anterior a la reconciliación (commit `7537e34`), por eso en sus logs no aparece `order_synced`.

5. Límites conocidos

1. **Stock descontado sin pedido:** si inventory responde después de que orders dejó de esperar (3 s), el stock puede quedar descontado sin que exista el pedido. Arreglarlo requiere reservas con ID propio. Queda como candidato a incidencia en la U3.
2. **Cola de despacho por fecha de creación:** la cola ordena por la fecha en que se creó el pedido, no por la del pago.
3. **Devolución de stock sin reintento:** si inventory no responde al anular, el stock no se devuelve solo; queda `stock_devuelto: false` para corregirlo a mano.
4. **Proveedor simulado:** en la U3 se prueba el contrato simulado, no una integración real con Webpay.

M4. Registro de stakeholders (parte 1)

La parte 2 (matriz de permisos por rol, servicio y ruta) va en `docs/m4-permisos.md` y depende del PR B y el PR C.

1. Registro

ID	Stakeholder	Tipo	Interés en el sistema	Rol en el sistema	¿Permisos operativos?	Cómo recibe la información
SH 01	Visitante	Persona externa	Ver productos y precios antes de registrarse	Anónimo (sin cuenta)	Solo lectura del catálogo	Catálogo público (S01, S02)
SH 02	Cliente	Persona externa	Comprar online, pagar, seguir y anular sus pedidos	CLIENTE	Sí, solo sobre sus propios datos y pedidos	Pantallas del cliente y boleta por correo
SH 03	Administrador	Funcionario	Mantener productos, usuarios y clientes, y apoyar en ventas y anulaciones	ADMIN	Sí, el más amplio	Panel interno
SH 04	Dueño	Funcionario	Saber cuánto se vende por período	DUENO	Solo consulta del reporte de ventas	Reporte de ventas (S19)
SH 05	Cajero virtual	Funcionario	Asociar las ventas web a una caja (RF4)	CAJERO	Sí: orders le asigna cada pedido web con `cajero_id` (S11). La boleta digital (RF8) es brecha declarada	Pedidos asignados en orders
SH 06	Encargado de despacho	Funcionario	Ver los pedidos pagados en orden de llegada y marcar su avance	DESPACHO	Sí, solo sobre el estado de despacho	Cola de despacho (S14, S15)
SH 07	Cocina	Funcionarios	Preparar los pedidos en el orden en que llegan	Ninguno	**No**	Orden de despacho impresa que le entrega DESPACHO (RF13)
SH 08	Choferes	Funcionarios	Entregar los pedidos dentro del radio de 3 km	Ninguno	**No**	Orden de despacho impresa con dirección y teléfono
SH 09	Proveedor de pagos	Sistema externo	Cobrar y avisar el resultado del pago	Proveedor (secreto propio)	Solo puede enviar avisos de pago (S17)	Llamada al servicio S17
SH 10	API de mensajería	Sistema externo	Validar correos y enviar boletas (RF8, RF11)	Ninguno	**No**	Simulada en la U2 (brecha declarada)
SH 11	Servicios internos	Sistemas propios	Cumplir cada uno su parte (inventory, orders, clients, payments, auth, GraphQL)	Servicio, con secreto propio en Vault	Solo sobre sus propios datos	Llamadas internas a través de la red privada

2. Stakeholders sin permisos operativos

ID	Stakeholder	Por qué no tiene permisos	Qué información recibe y cómo
SH 07	Cocina	No necesita usar el sistema para preparar	La orden de despacho impresa, con los productos y cantidades, en orden de llegada

ID	Stakeholder	Por qué no tiene permisos	Qué información recibe y cómo
SH 08	Choferes	Solo entregan. Darles una cuenta los expondría a datos de otros clientes	La orden de despacho impresa con nombre, dirección, teléfono y productos. Sin RUN, correo ni montos
SH 10	API de mensajería	Es un sistema externo que solo recibe solicitudes	El correo a validar o la boleta a enviar. Simulada en la U2

3. Reglas que salen de este registro

1. Un CLIENTE solo ve y modifica sus propios datos y pedidos. Lo revisa orders, no solo el gateway.
2. Anular un pedido no lo borra: cambia su estado a ANULADO y conserva el motivo.
3. DUENO ve totales, no datos personales de clientes.
4. DESPACHO ve datos de contacto solo de pedidos pagados.
5. El visitante no es un rol: es una solicitud sin sesión.

M4. Matriz de permisos (parte 2)

La parte 1 (registro de stakeholders) está en `docs/m4-stakeholders.md`.

Fuente: código del gateway, auth, inventory, orders, payments y graphql-gateway en `main` (commit `5ee71b8`). S09 y S10 (clients) no están implementados en la U2: el registro y el perfil quedan en el front con localStorage.

1. Cómo se decide un permiso

1. **Gateway:** la ruta tiene que estar en la lista blanca (`RUTAS` en `backend/gateway/gateway.py`). Si la ruta pide sesión, el gateway valida la cookie con auth (`/introspect`). Si pide un rol, el gateway lo revisa. Después reenvía la identidad en `X-Authenticated-User` y `X-Authenticated-Roles`, y descarta los que haya mandado el navegador.
2. **Servicio:** cada servicio exige el secreto del gateway (`X-Gateway-Secret`) y vuelve a revisar el rol o la propiedad del recurso. La regla de propiedad (un cliente solo ve lo suyo) la revisan orders y payments, no solo el gateway.

Códigos: ✅ permitido, **401** sin sesión o sesión inválida, **403** rol sin permiso, **404** recurso ajeno o inexistente (no se confirma que exista), **409** el estado no lo permite.

2. Matriz por servicio y rol

ID	Verbo y ruta	Servicio	Anónimo	CLIENTE	ADMIN	DUENO	CAJERO	DESPACHO	Condición	Dónde se verifica
S01	`GET /api/v1/productos`	inventory	✅	✅	✅	✅	✅	✅	Ninguna	Gateway (ruta anónima) e inventory (secreto del gateway)
S02	`GET /api/v1/productos/{id}`	inventory	✅	✅	✅	✅	✅	✅	Ninguna	Gateway e inventory
S03	`POST /api/v1/productos`	inventory	401	403	✅	403	403	403	Solo ADMIN	Gateway (rol) e inventory (`verify_admin`)
S04	`PUT /api/v1/productos/{id}`	inventory	401	403	✅	403	403	403	Solo ADMIN	Gateway e inventory
S05	`DELETE /api/v1/productos/{id}`	inventory	401	403	✅	403	403	403	Solo ADMIN	Gateway e inventory
S06	`POST /api/v1/auth/login`	auth	✅	—	—	—	—	—	Correo y clave correctos	auth (bcrypt)
S07	`POST /api/v1/auth/logout`	auth	✅	✅	✅	✅	✅	✅	Sin sesión responde igual (no hay nada que cerrar)	Gateway y auth
S08	`GET /api/v1/auth/me`	auth	401	✅	✅	✅	✅	✅	Sesión vigente	Gateway (`/introspect` en auth)

ID	Verbo y ruta	Servicio	Anónimo	CLIENTE	ADMIN	DUENO	CAJERO	DESPACHO	Condición	Dónde se verifica
S09	`POST /api/v1/clientes`	clients	✅	—	—	—	—	—	Registro anónimo con límite de 10 por minuto por IP; RUN y correo únicos; la cuenta queda inactiva hasta verificar	Gateway (límite) y clients (validación); la credencial la crea auth con el secreto propio de clients
S09V	`POST /api/v1/clientes/{id}/verificar`	clients	✅	—	—	—	—	—	Código de 6 dígitos correcto; mismo error si la cuenta no existe	Gateway (límite) y clients (hash del código)
S10	`GET /api/v1/clientes/me`	clients	401	Solo el suyo	403	403	403	403	El id sale de la sesión, no de la URL: no existe forma de pedir otro perfil	Gateway (rol) y clients (`cliente_autenticado`)
S11	`POST /api/v1/pedidos`	orders	401	✅	403	403	403	403	Solo CLIENTE, con `Idempotency-Key`	Gateway (rol) y orders (rol)
S12	`GET /api/v1/pedidos/{id}`	orders	401	Solo los suyos (ajeno 404)	✅ todos	404	404	Solo pedidos vendidos (otros 404)	Propiedad del pedido	Gateway (sesión) y orders (`puede_ver`)
S13	`POST /api/v1/pedidos/{id}/anular`	orders	401	Solo los suyos (ajeno 404)	✅	404	404	404	Pedido en PENDIENTE_PAGO (si no, 409) y motivo obligatorio	Gateway (sesión) y orders (propiedad y estado)
S14	`GET /api/v1/pedidos`	orders	401	403	✅ todos	403	403	Solo vendidos (pedir otro estado da 403)	Rol y estado pedido	Gateway (rol) y orders (filtro por rol)
S15	`PATCH /api/v1/pedidos/{id}/estado`	orders	401	403	✅	403	403	✅	Solo PAGADO → EN_DESPACHO → ENTREGADO (otra transición 409)	Gateway (rol) y orders (rol y transición)
S16	`POST /api/v1/pagos`	payments	401	Solo pedidos suyos (ajeno 404)	403	403	403	403	Pedido en PENDIENTE_PAGO (si no, 409), con `Idempotency-Key`	Gateway (rol) y payments (rol y propiedad)
S17	`POST /api/v1/pagos/aviso`	payments	Solo el proveedor con su secreto (sin secreto 401)						Header `X-Proveedor-Secret` válido	payments (`verify_proveedor`)
S18	`POST /api/v1/graphql`	graphql-gateway	401	Solo sus pedidos (ajeno `null`)	Error `FORBIDDEN`	Error `FORBIDDEN`	Error `FORBIDDEN`	Error `FORBIDDEN`	Queries solo para CLIENTE; el campo `Producto.stock` solo para ADMIN (`FORBIDDEN_FIELD`)	Gateway (sesión), graphql-gateway (rol y campo) y orders (propiedad)
S19	`GET /api/v1/reportes/ventas`	orders	401	403	✅	✅	403	403	Solo DUENO o ADMIN	Gateway (rol) y orders (rol)

3. Operaciones internas (no expuestas por el gateway)

No están en la lista blanca, así que desde fuera responden 404. Solo las puede llamar el servicio que tiene el secreto de esa integración.

Operación	Quién la llama	Secreto que exige	Para qué
`POST /interno/reservas` (inventory)	orders	`ORDERS_INVENTORY_SECRET`	Descontar stock y obtener el precio vigente al crear un pedido
`POST /interno/devoluciones` (inventory)	orders	`ORDERS_INVENTORY_SECRET`	Devolver el stock al anular un pedido
`GET /interno/pedidos/{id}` (orders)	payments	`PAYMENTS_ORDERS_SECRET`	Consultar dueño, estado y total del pedido (el monto nunca lo manda el navegador)
`POST /interno/pedidos/{id}/pagado` (orders)	payments	`PAYMENTS_ORDERS_SECRET`	Marcar el pedido como PAGADO y emitir la boleta. Solo payments puede hacerlo
`POST /introspect` y `POST /logout` (auth)	gateway	`auth_introspection_secret`	Validar y cerrar sesiones
`POST /graphql` (graphql-gateway)	gateway	`graphql_shared_secret`	Ejecutar las queries de S18 con la identidad ya validada
`GET /pedidos` y `GET /pedidos/{id}` (orders)	graphql-gateway	`GRAPHQL_ORDERS_SECRET`	Leer los pedidos del cliente. Con este secreto orders no acepta escrituras
`POST /interno/productos/lote` (inventory)	graphql-gateway	`GRAPHQL_INVENTORY_SECRET`	Leer varios productos en una sola llamada (batching)

4. Reglas que salen de la matriz

1. **Propiedad:** un CLIENTE solo ve, anula y paga sus propios pedidos. Lo revisan orders (S12, S13, y S18 a través de GraphQL) y payments (S16), no solo el gateway. Un pedido ajeno responde 404 por REST y `null` por GraphQL, igual que uno inexistente.
2. **Anular no es borrar:** el pedido pasa a ANULADO, guarda el motivo y el historial, y devuelve el stock.
3. **PAGADO lo decide solo payments**, mediante una operación interna de orders. Ni el cliente ni DESPACHO pueden poner un pedido en PAGADO.
4. **DESPACHO** solo ve pedidos vendidos (PAGADO, EN_DESPACHO o ENTREGADO), y la dirección solo de esos pedidos.
5. **DUENO** solo ve el reporte de ventas (totales por producto, sin datos personales de clientes).
6. **CAJERO** no tiene rutas operativas en la U2: orders le asigna cada pedido web con `cajero_id` (R04). Puede iniciar y cerrar sesión.
7. **Defensa en profundidad:** las rutas con rol se revisan dos veces (gateway y servicio). Si alguien llegara a un servicio sin pasar por el gateway, no tendría el secreto y recibiría 403.

M5. API Gateway, fronteras de confianza y recorridos

Diagrama de clases: `docs/uml-gateway.drawio` (se abre en app.diagrams.net).

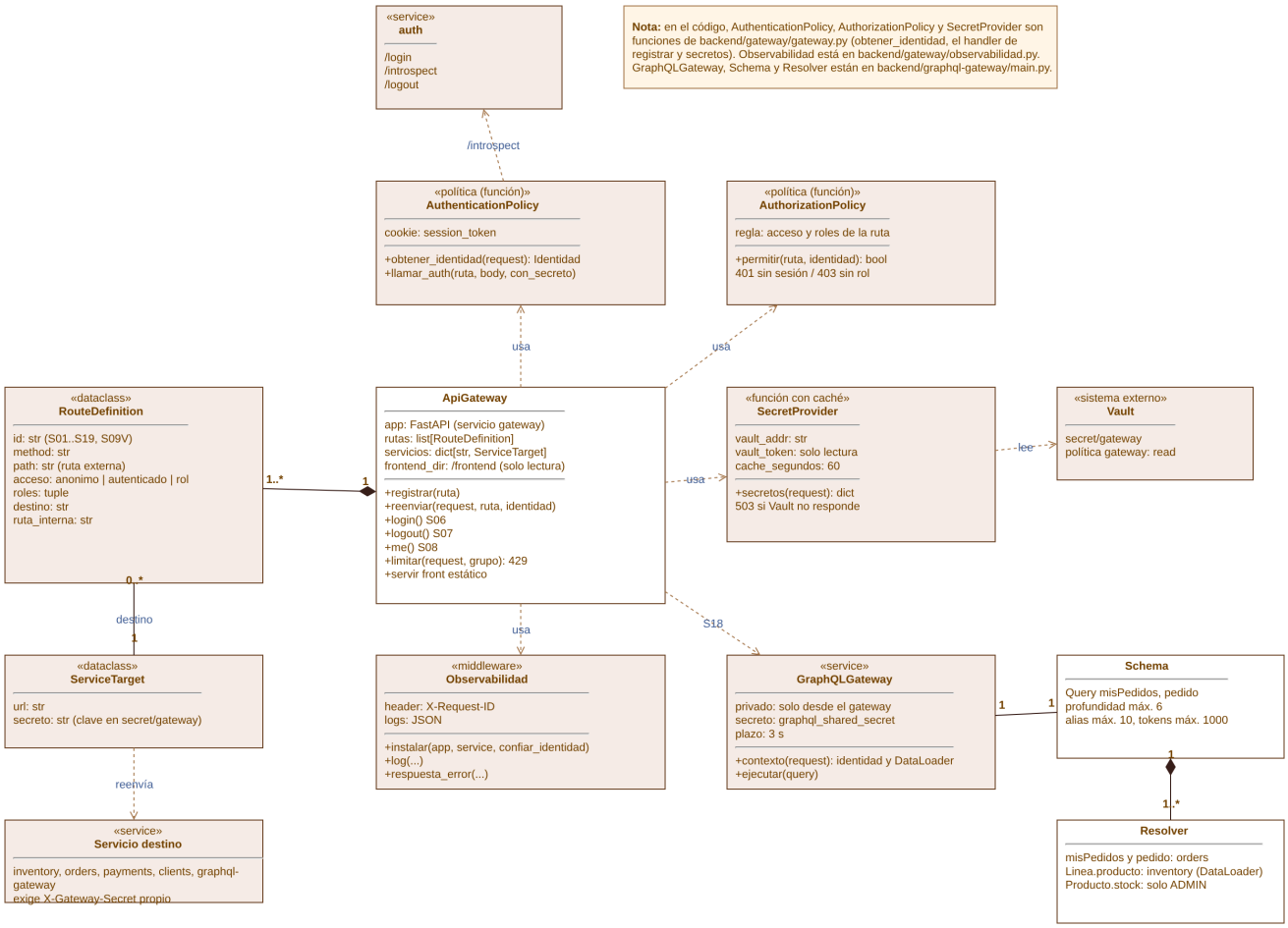


Diagrama UML del API Gateway y sus políticas (fuente editable: docs/uml-gateway.drawio)

Fuente: `backend/gateway/gateway.py`, `backend/gateway/observabilidad.py`, `backend/graphql-gateway/main.py` y `docker-compose.yml` en `main` (commit `5ee71b8`). El detalle de GraphQL está en `docs/m8-graphql.md`.

1. Un solo gateway

Chocomanía usa **un único API Gateway**. Se eligió uno solo porque:

- Hay un solo tipo de cliente (el front web) y un solo punto de entrada público.
- La carga proyectada es baja (menos de 25 solicitudes por segundo en el peak, ver `docs/m9-carga.md`).
- Un solo lugar concentra la lista blanca, la validación de sesión, el `X-Request-ID` y la lectura de secretos, así que es más fácil de revisar y probar.

El GraphQL Gateway no es un segundo punto de entrada: es privado (sin puerto publicado) y solo se llega a él a través del API Gateway, por S18. El API Gateway también sirve los archivos del front en el mismo origen (`http://localhost:8000`), así que el navegador no necesita CORS.

2. Clases del diagrama

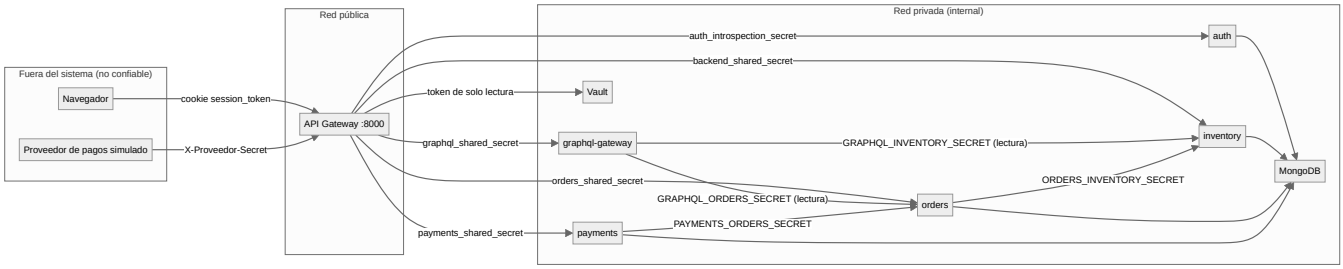
Clase	Qué representa	Dónde está en el código
ApiGateway	La aplicación FastAPI del gateway: registra las rutas de la lista blanca, las reenvía y sirve el front estático	`gateway.py` (`app`, `registrar`, `reenviar`, `login`, `logout`, `me` y el montaje de `FRONTEND_DIR`)
RouteDefinition	Una ruta permitida: ID del M1, verbo, ruta externa, tipo de acceso, roles, destino y ruta interna	`gateway.py` (`@dataclass RouteDefinition`, lista `RUTAS`)
ServiceTarget	Un servicio destino: URL interna y nombre de su secreto en Vault	`gateway.py` (`@dataclass ServiceTarget`, diccionario `SERVICIOS`)
AuthenticationPolicy	Valida la cookie `session_token` con auth (`/introspect`) y arma la identidad	`gateway.py` (funciones `obtener_identidad` y `llamar_auth`)
AuthorizationPolicy	Decide si la identidad cumple el acceso de la ruta: anónimo, autenticado o rol	`gateway.py` (handler dentro de `registrar`)
SecretProvider	Lee los secretos de `secret/gateway` en Vault con un token de solo lectura, con caché de 60 s	`gateway.py` (función `secretos`)
Observabilidad	Middleware de `X-Request-ID`, logs JSON y formato único de error	`observabilidad.py` (`instalar`, `log`, `respuesta_error`)
GraphQLGateway	Servicio privado que exige su secreto, arma el contexto con la identidad y aplica los límites	`graphql-gateway/main.py` (`app`, `contexto`, `GraphQLRouter`)
Schema	Queries `misPedidos` y `pedido`, tipos y límites de profundidad (6), alias (10) y tokens (1000)	`graphql-gateway/main.py` (`Query`, `SchemaConLogs`)
Resolver	Resuelve cada campo; `Linea.producto` usa un DataLoader para pedir todos los productos en una sola llamada	`graphql-gateway/main.py` (`mis_pedidos`, `pedido`, `Linea.producto`, `cargador_productos`)

En el código, AuthenticationPolicy, AuthorizationPolicy y SecretProvider son **funciones**, no clases. En el diagrama se muestran como clases para separar sus responsabilidades.

3. Clasificación de rutas

Tipo	Rutas	Qué exige el gateway
Anónimas	S01, S02 (catálogo), S06 (login), S07 (logout, con o sin sesión)	Nada. Igual pasan por la lista blanca, la validación de parámetros y el `X-Request-ID`
Externa con secreto propio	S17 (aviso del proveedor de pagos)	Nada en el gateway; payments exige `X-Proveedor-Secret`
Autenticadas	S08, S12, S13, S18	Sesión vigente. La propiedad del pedido la revisa orders; en S18, el rol y el campo los revisa graphql-gateway
Por rol	S03 a S05 (ADMIN), S11 y S16 (CLIENTE), S14 y S15 (DESPACHO o ADMIN), S19 (DUENO o ADMIN)	Sesión vigente y rol permitido. El servicio vuelve a revisar el rol
Front estático	`GET /`, `/*.html`, `/js/*`, `/css/*`, `/img/*`	Nada: son archivos públicos. Solo se sirven archivos de la carpeta `frontend/`
Internas (no expuestas)	`/interno/*` de inventory y orders, `/introspect` y `/logout` de auth, `/graphql` de graphql-gateway, `/health` de cada servicio	No están en la lista blanca: desde fuera responden 404
Anónimas con límite estricto	S09, S09V (clients) y S06 (login)	Límite por IP en el gateway (10 y 20 por minuto) contra registros masivos y fuerza bruta; 429 `RATE_LIMITED`

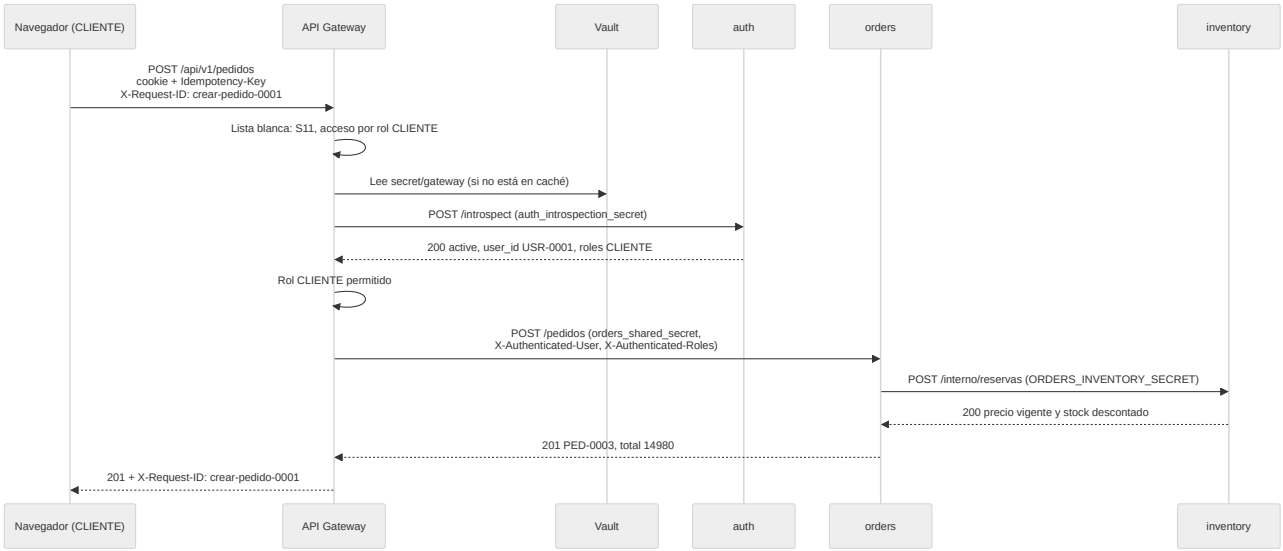
4. Fronteras de confianza



Frontera	Qué no se confía	Cómo se controla
1. Navegador → gateway	Nada de lo que manda el navegador	Lista blanca de rutas y verbos; parámetros validados (`^[A-Za-z0-9_-]{1,64}$`); sesión validada con auth en cada solicitud; los `X-Authenticated-*` del navegador se descartan; `X-Request-ID` se acepta solo si es válido; solo se reenvían `content-type`, `idempotency-key` y `x-proveedor-secret`
2. Proveedor → gateway → payments	El aviso de pago	payments revisa `X-Proveedor-Secret` y procesa cada aviso una sola vez
3. Gateway → servicios internos	Que alguien llame a un servicio sin pasar por el gateway	Red privada sin acceso desde fuera; cada servicio exige su propio `X-Gateway-Secret`; solo con ese secreto confía en `X-Authenticated-User` y `X-Authenticated-Roles`
4. Servicio → servicio	Que un servicio use una operación que no le corresponde	Secretos distintos por integración: orders → inventory, payments → orders y graphql-gateway → orders e inventory. Los de graphql-gateway solo sirven para leer
5. Gateway → Vault	Que el gateway lea más de lo que necesita	Token con política de solo lectura sobre `secret/gateway`
6. Servicios → MongoDB	—	Cada servicio usa su propia base, pero todos con el usuario raíz (brecha declarada en el M6)

5. Recorrido de una solicitud autorizada

Caso: un CLIENTE crea un pedido (S11). Es la ejecución real guardada en `evidencias/u2/pedido-idempotente/logs.txt` (rama `feat/orders`), con `request_id` **`crear-pedido-0001`**.



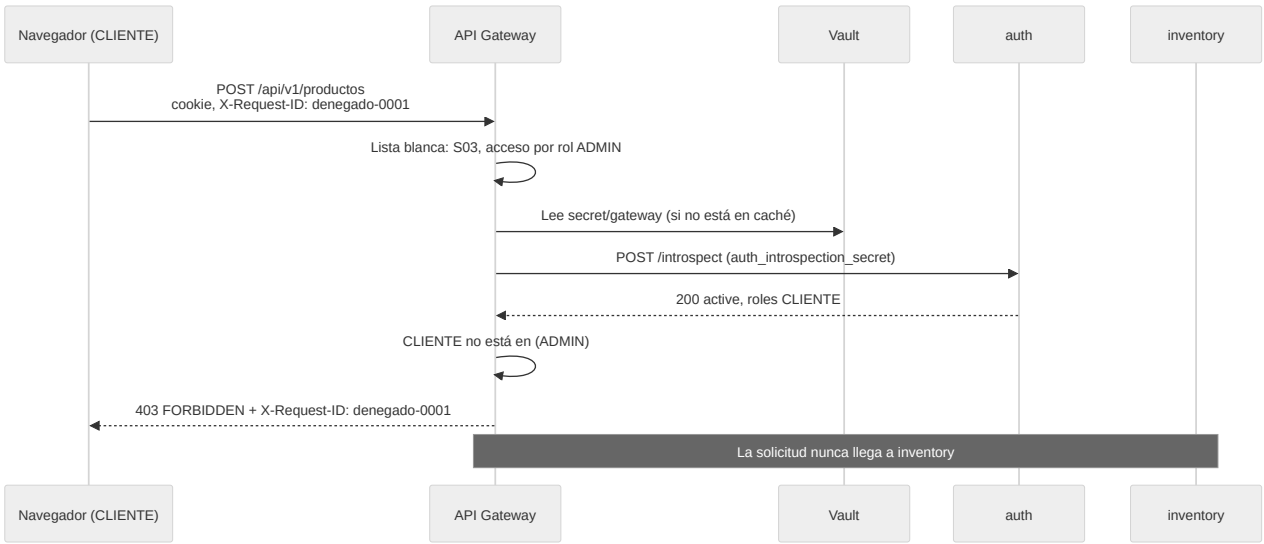
Las líneas de log, todas con `request_id: crear-pedido-0001`, en orden:

Hora (UTC)	Servicio	Evento	Detalle
13:51:34.548	gateway	`secret_loaded`	Lee `secret/gateway`; registra solo los nombres de las claves

Hora (UTC)	Servicio	Evento	Detalle
13:51:34.611	auth	`request`	`POST /introspect` → 200
13:51:35.230	inventory	`request`	`POST /interno/reservas` → 200
13:51:35.260	orders	`order_created`	`PED-0003`, `USR-0001`, total 14980
13:51:35.263	orders	`request`	`POST /pedidos` → 201
13:51:35.266	gateway	`request`	`POST /api/v1/pedidos` → 201, `USR-0001`, 800 ms

6. Recorrido de una solicitud denegada

Caso: un CLIENTE intenta crear un producto (S03, solo ADMIN). Recorrido según el código; todavía no hay una ejecución guardada como evidencia (corresponde a la prueba P03 del protocolo).



Lo que se espera ver:

- Respuesta: `403` con `{ "error": { "code": "FORBIDDEN", "message": "No tienes permiso para esta operacion", "request_id": "denegado-0001" } }`.
- Logs con el mismo `request_id`: auth (`/introspect` → 200) y gateway (`POST /api/v1/productos` → 403). **Ninguna línea de inventory.**
- Si el navegador manda además `X-Authenticated-Roles: ADMIN` para engañar al gateway, el resultado es el mismo, porque ese header se descarta.

Otras denegaciones que corta el gateway antes de llegar al servicio: sin cookie (401 `UNAUTHENTICATED`), sesión vencida (401 `SESSION_INVALID`), ruta o verbo fuera de la lista blanca (404 o 405) e ID con formato inválido (400 `INVALID_ID`).

M6. Matriz de rutas e inventario de secretos

Fuente: `backend/gateway/gateway.py`, `backend/graphql-gateway/main.py`, `docker-compose.yml`, `scripts/vault-init.sh` y `.env.example` en `main` (commit `5ee71b8`). Clients (S09, S10) no está implementado en la U2.

1. Un solo gateway

Hay un único API Gateway (`gateway`, puerto 8000). Es el único contenedor en la red pública; todos los demás (auth, inventory, orders, payments, graphql-gateway, Vault y Mongo) están en la red privada `privada`, marcada como `internal` en `docker-compose.yml`, sin entrada ni salida desde fuera. Con el volumen del caso (ver `docs/m9-carga.md`) un gateway basta.

2. Controles comunes a todas las rutas

Control	Cómo funciona
Lista blanca	Solo existen las rutas de la tabla `RUTAS` y los archivos estáticos del front. Cualquier otra ruta o verbo responde 404 o 405
Validación de parámetros	Los `{id}` de la ruta deben cumplir `^[A-Za-z0-9_-]{1,64}$`; si no, 400 `INVALID_ID`
Sesión	Cookie `session_token` (`httponly`, `samesite=lax`), validada en cada solicitud con `/introspect` de auth
Identidad hacia los servicios	El gateway pone `X-Authenticated-User` y `X-Authenticated-Roles` y descarta los que mande el navegador
Headers reenviados	Solo `content-type`, `idempotency-key` y `x-proveedor-secret`, además de los que agrega el gateway
Correlación	`X-Request-ID` (8 a 64 caracteres válidos se reutiliza; si no, se genera) en la respuesta y en los logs
Secreto por servicio	Cada servicio interno exige `X-Gateway-Secret` con el valor de su propia integración
Plazos	5 s hacia los servicios (`UPSTREAM_TIMEOUT_SECONDS`): 504 si no responden, 502 si no están disponibles
Límite de solicitudes	Ventana deslizante por IP en el gateway: login 20/min, registro y verificación 10/min, resto 12.000/min. Responde 429 `RATE_LIMITED` (prueba `test_zz_limite.py`)

3. Matriz de rutas

ID	Gateway	Verbo	Ruta externa	Acceso y rol	Destino (servicio y ruta interna)	Controles propios	Referencia de credencial
S01	gateway	GET	`/api/v1/productos`	Anónimo	inventory `/productos`	Paginación y filtro por categoría validados en inventory	`secret/gateway#backend_shared_secret`
S02	gateway	GET	`/api/v1/productos/{id}`	Anónimo	inventory `/productos/{id}`	ID validado	`secret/gateway#backend_shared_secret`
S03	gateway	POST	`/api/v1/productos`	ADMIN	inventory `/productos`	Rol revisado en gateway e inventory	`secret/gateway#backend_shared_secret`
S04	gateway	PUT	`/api/v1/productos/{id}`	ADMIN	inventory `/productos/{id}`	Rol revisado en gateway e inventory	`secret/gateway#backend_shared_secret`
S05	gateway	DELETE	`/api/v1/productos/{id}`	ADMIN	inventory `/productos/{id}`	Rol revisado en gateway e inventory	`secret/gateway#backend_shared_secret`

ID	Gateway	Verbo	Ruta externa	Acceso y rol	Destino (servicio y ruta interna)	Controles propios	Referencia de credencial
S06	gateway	POST	`/api/v1/auth/login`	Anónimo	auth `/login`	Misma respuesta si el correo no existe o la clave está mal; deja la cookie	Ninguna (auth `/login` solo es alcanzable desde la red privada)
S07	gateway	POST	`/api/v1/auth/logout`	Con o sin sesión	auth `/logout`	Borra la sesión en auth y la cookie	`secret/gateway#auth_introspection_secret`
S08	gateway	GET	`/api/v1/auth/me`	Autenticado	auth `/introspect`	No reenvía el token a otros servicios	`secret/gateway#auth_introspection_secret`
S09	gateway	POST	`/api/v1/clientes`	Anónimo	clients `/clientes`	Límite 10/min por IP; clients valida RUN (módulo 11), correo y teléfono; la clave va a auth y solo se guarda su hash	`secret/gateway#clients_shared_secret`
S09V	gateway	POST	`/api/v1/clientes/{id}/verificar`	Anónimo	clients `/clientes/{id}/verificar`	ID validado; límite 10/min por IP; código guardado como hash SHA-256	`secret/gateway#clients_shared_secret`
S10	gateway	GET	`/api/v1/clientes/me`	CLIENTE	clients `/clientes/me`	Identidad del gateway; devuelve solo los campos del perfil (sin clave, código ni fecha de nacimiento)	`secret/gateway#clients_shared_secret`
S11	gateway	POST	`/api/v1/pedidos`	CLIENTE	orders `/pedidos`	`Idempotency-Key` obligatorio; precio desde inventory	`secret/gateway#orders_shared_secret`
S12	gateway	GET	`/api/v1/pedidos/{id}`	Autenticado	orders `/pedidos/{id}`	Propiedad revisada en orders	`secret/gateway#orders_shared_secret`
S13	gateway	POST	`/api/v1/pedidos/{id}/anular`	Autenticado	orders `/pedidos/{id}/anular`	Propiedad y estado revisados en orders	`secret/gateway#orders_shared_secret`
S14	gateway	GET	`/api/v1/pedidos`	DESPACHO, ADMIN	orders `/pedidos`	DESPACHO filtrado a pedidos vendidos	`secret/gateway#orders_shared_secret`
S15	gateway	PATCH	`/api/v1/pedidos/{id}/estado`	DESPACHO, ADMIN	orders `/pedidos/{id}/estado`	Solo transiciones de despacho	`secret/gateway#orders_shared_secret`
S16	gateway	POST	`/api/v1/pagos`	CLIENTE	payments `/pagos`	`Idempotency-Key` obligatorio; monto desde orders	`secret/gateway#payments_shared_secret`
S17	gateway	POST	`/api/v1/pagos/aviso`	Proveedor (sin sesión)	payments `/pagos/aviso`	`X-Proveedor-Secret` revisado en payments; aviso repetido no cambia nada	`secret/gateway#payments_shared_secret` y `PAYMENT_PROVIDER_SECRET`
S18	gateway	POST	`/api/v1/graphql`	Autenticado	graphql-gateway `/graphql`	Rol y campo revisados en graphql-gateway; límites de profundidad, alias, tokens y paginación; solo POST	`secret/gateway#graphql_shared_secret`
S19	gateway	GET	`/api/v1/reportes/ventas`	DUENO, ADMIN	orders `/reportes/ventas`	Rol revisado en gateway y orders	`secret/gateway#orders_shared_secret`
—	gateway	GET	`/` y archivos del front (`index.html`, `js/`, `css/`, `img/`)	Anónimo	Carpeta `frontend/`, montada en solo lectura	Solo archivos de esa carpeta; va después de las rutas de la API, así que no las tapa	Ninguna

Filas GraphQL

Detalle en `docs/m8-graphql.md`.

Operación	Tipo	Campos autorizados	Resolver	Subsistemas
`misPedidos(pagina, porPagina)`	Query	Pedidos del CLIENTE con líneas, boleta, despacho (sin teléfono) y producto; `stock` solo ADMIN	`Query.misPedidos` + `Linea.producto` (DataLoader)	orders (`GRAPHQL_ORDERS_SECRET`) e inventory (`GRAPHQL_INVENTORY_SECRET`)
`pedido(id)`	Query	Un pedido propio; ajeno o inexistente devuelve `null`	`Query.pedido` + `Linea.producto`	orders e inventory

4. Inventario de secretos

Solo se registran **referencias**, nunca valores. Los valores están en el archivo `.env` de cada computador, que está en `.gitignore`; `.env.example` los lista vacíos.

Referencia	Tipo	Emisor	Consumidor	Finalidad	Ubicación	Alcance	Rotación
`VAULT_ROOT_TOKEN`	Token raíz de Vault (modo dev)	Vault al iniciar	`vault-init`	Cargar los secretos y crear el token del gateway	`.env`	Total sobre Vault	Manual, al recrear el entorno
`GATEWAY_VAULT_TOKEN`	Token de Vault	`vault-init`	gateway	Leer sus secretos	`.env` (variable `VAULT_TOKEN` del gateway)	Solo lectura de `secret/gateway` (política `gateway`)	Revocar y crear otro con `vault-init`
`secret/gateway #backend_shared_secret` (`GATEWAY_INVENTORY_SECRET`)	Secreto compartido	`.env` → `vault-init`	gateway → inventory	Que inventory solo acepte solicitudes del gateway	Vault (lado gateway) y `.env` (lado inventory)	S01 a S05	Cambiar en `.env`, recargar Vault y reiniciar inventory
`secret/gateway #auth_introspection_secret` (`AUTH_INTROSPECTION_SECRET`)	Secreto compartido	`.env` → `vault-init`	gateway → auth	Validar y cerrar sesiones	Vault (lado gateway) y `.env` (lado auth)	`/introspect` y `/logout`	Igual que el anterior
`secret/gateway #orders_shared_secret` (`GATEWAY_ORDERS_SECRET`)	Secreto compartido	`.env` → `vault-init`	gateway → orders	Que orders solo acepte solicitudes del gateway	Vault (lado gateway) y `.env` (lado orders)	S11 a S15 y S19	Igual que el anterior
`secret/gateway #payments_shared_secret` (`GATEWAY_PAYMENTS_SECRET`)	Secreto compartido	`.env` → `vault-init`	gateway → payments	Que payments solo acepte solicitudes del gateway	Vault (lado gateway) y `.env` (lado payments)	S16 y S17	Igual que el anterior
`secret/gateway #graphql_shared_secret` (`GATEWAY_GRAPHQL_SECRET`)	Secreto compartido	`.env` → `vault-init`	gateway → graphql-gateway	Que graphql-gateway solo acepte solicitudes del gateway	Vault (lado gateway) y `.env` (lado graphql-gateway)	S18	Igual que el anterior
`secret/gateway #clients_shared_secret` (`GATEWAY_CLIENTS_SECRET`)	Secreto compartido	`.env` → `vault-init`	gateway → clients	Que clients solo acepte solicitudes del gateway	Vault (lado gateway) y `.env` (lado clients)	S09, S09V y S10	Cambiar en `.env`, recargar Vault y reiniciar clients
`CLIENTS_AUTH_SECRET`	Secreto compartido	`.env`	clients → auth	Crear, activar o revertir credenciales	`.env` (no está en Vault)	Solo `/usuarios` de auth y siempre con rol CLIENTE: no puede crear personal ni administradores	Cambiar en `.env` y reiniciar clients y auth

Referencia	Tipo	Emisor	Consumidor	Finalidad	Ubicación	Alcance	Rotación
`GRAPHQL_ORDERS_SECRET`	Secreto compartido (solo lectura)	`.env`	graphql-gateway → orders	Leer pedidos	`.env` (no está en Vault)	`GET /pedidos` y `GET /pedidos/{id}`	Cambiar en `.env` y reiniciar graphql-gateway y orders
`GRAPHQL_INVENTORY_SECRET`	Secreto compartido (solo lectura)	`.env`	graphql-gateway → inventory	Leer productos por lote	`.env` (no está en Vault)	`POST /interno/productos/lote`	Cambiar en `.env` y reiniciar graphql-gateway e inventory
`ORDERS_INVENTORY_SECRET`	Secreto compartido	`.env`	orders → inventory	Reservar y devolver stock	`.env` (no está en Vault)	`/interno/reservas` y `/interno/devoluciones`	Cambiar en `.env` y reiniciar orders e inventory
`PAYMENTS_ORDERS_SECRET`	Secreto compartido	`.env`	payments → orders	Consultar el pedido y marcarlo pagado	`.env` (no está en Vault)	`/interno/pedidos/*`	Cambiar en `.env` y reiniciar payments y orders
`PAYMENT_PROVIDER_SECRET`	Secreto del proveedor	`.env` (lo comparte el proveedor simulado)	proveedor → payments	Autenticar el aviso de pago	`.env` (no está en Vault)	S17	Cambiar en `.env` y en el proveedor, reiniciar payments
`MONGO_ROOT_USER` / `MONGO_ROOT_PASSWORD`	Usuario y clave de base de datos	`.env`	auth, inventory, orders, payments, seed y check (graphql-gateway no usa Mongo)	Acceso a Mongo	`.env`	Usuario raíz: acceso a todas las bases	Manual, al recrear el volumen
`session_token`	Token de sesión	auth al iniciar sesión	navegador → gateway	Mantener la sesión	Cookie `httponly` en el navegador; en auth solo su hash SHA-256	Una sesión	Expira por `AUTH_TOKEN_TTL` (900 s) o al cerrar sesión

Las contraseñas de los usuarios no son secretos de integración: se guardan solo como hash bcrypt (ver `docs/m7-datos-sensibles.md`).

5. Qué hacer si se filtra una clave

1. Registrar la incidencia en `docs/incidencias.md` (sin escribir el valor filtrado).
2. Dar el valor por quemado: no se vuelve a usar nunca.
3. Generar un valor nuevo en `.env`.
4. Si es un secreto que está en Vault, volver a correr `vault-init` para cargarlo y reiniciar el servicio del otro lado. El gateway toma el valor nuevo en máximo 60 segundos, que es lo que dura su caché (`VAULT_CACHE_SECONDS`).
5. Si es un secreto que solo está en `.env`, reiniciar los dos servicios que lo comparten.
6. Si es el token del gateway, revocarlo en Vault y crear otro.
7. Si es una sesión, cerrarla con S07 o esperar que expire.
8. Revisar los logs con `scripts/logs.sh` para ver si se usó.

Valores que ya se dan por quemados (quedaron en el historial de git en semanas anteriores): los secretos y el token de laboratorio de las semanas 8 y 9. No se reutilizan.

6. Qué pasa si Vault no responde

- El gateway sigue funcionando con los secretos en caché durante 60 segundos.
- Después responde **503** `SECRETS_UNAVAILABLE` a todas las rutas de la API, incluido el catálogo anónimo (también necesita el secreto de inventory), y registra `vault_unavailable` en su log. Las páginas del front se siguen cargando, porque no usan secretos, pero muestran el error al llamar a la API.
- Los servicios internos no dependen de Vault en la U2, porque reciben sus secretos por variables de entorno.
- Vault corre en modo dev: si se reinicia pierde los secretos, y hay que volver a correr `vault-init` (lo hace `docker compose up`).

7. Revisión de secretos en el repo, capturas y logs

Dónde	Resultado
`.env`	En `.gitignore`, no está en el repo
`.env.example`	Solo nombres, todos vacíos
Logs	El gateway registra `secret_loaded` con los nombres de las claves, nunca los valores. Los logs no incluyen tokens, cookies, contraseñas ni correos
Evidencias (`evidencias/u2/`)	Revisadas: sin secretos ni tokens; solo datos ficticios
`backend/gateway/README.md`	Tenía las instrucciones de la Semana 8 con secretos de laboratorio quemados. **Borrado** en INCR-24. Siguen en el historial de git; eran valores de laboratorio que no se usan en el stack actual
`frontend/js/login.js`	Solo muestra la cuenta de cliente de prueba (ficticia). Las cuentas del personal se sacaron del front y quedan solo en el README

8. Brechas declaradas

1. **Solo el gateway lee secretos desde Vault.** Los servicios internos reciben los suyos como variables de entorno desde `.env`, y los secretos entre servicios (`ORDERS_INVENTORY_SECRET`, `PAYMENTS_ORDERS_SECRET`, `PAYMENT_PROVIDER_SECRET`, `GRAPHQL_ORDERS_SECRET` y `GRAPHQL_INVENTORY_SECRET`) no están en Vault.
2. **Una sola credencial de Mongo para todo.** Cada servicio usa su propia base (`auth`, `inventory`, `orders`, `payments`), pero todos se conectan con el usuario raíz, así que la separación no la hace cumplir la base de datos.
3. **Vault en modo dev**, con datos en memoria y token raíz en `.env`. Sirve para el entorno local, no para producción.
4. **Límite de solicitudes en memoria del gateway:** sirve para un solo gateway; con varias réplicas tendría que ir en un almacén compartido (por ejemplo Redis).
5. **Cookie sin `secure`** en local (`COOKIE_SECURE=false`), porque no hay HTTPS.

M7. Datos sensibles

Estado: versión final, revisada contra el código de `main` (bdd26ff): auth, inventory, orders, payments, graphql-gateway, gateway y `frontend/js`. Las 6 pruebas de seguridad (sección 7) se ejecutaron el 08-11-2026 con pytest y pasaron todas (E07).

Actualización del 9 de noviembre: el servicio clients quedó implementado (S09, S09V y S10). RUN, dirección, teléfono, fecha de nacimiento y sexo se guardan en la base `clients`; la clave la recibe auth y guarda solo su hash bcrypt. S10 devuelve solo lo que el front necesita (sin fecha de nacimiento ni sexo). Las filas que digan "clients" deben leerse como implementadas.

1. Niveles de clasificación

Nivel	Qué significa	Ejemplo
Público	Lo puede ver cualquiera, sin sesión	Catálogo, precios
Interno	Solo funcionarios con rol	Cola de despacho, reporte de ventas
Personal	Identifica a una persona. Solo la persona y los roles que lo necesitan	Nombre, correo, dirección
Restringido	Personal y de mayor riesgo si se filtra. Se muestra enmascarado	RUN, fecha de nacimiento, sexo
Secreto	Nadie lo ve en texto plano, ni siquiera el administrador	Contraseñas, tokens de sesión, secretos entre servicios

2. Matriz de datos

Dato	Finalidad	Clasificación	Custodio	Roles que acceden	Protección	Retención	Estado en la U2
RUN	Identificar al cliente (RF5) y emitir la boleta	Restringido	clients	CLIENTE (el suyo), ADMIN	Enmascarado (`**.***.678-5`), único, validado con dígito verificador, nunca en logs	Mientras la cuenta esté activa	Implementado en clients (S09): único, validado con dígito verificador también en el servidor; S10 lo devuelve enmascarado
Nombre	Saludo, despacho y boleta	Personal	auth (usuarios)	El propio usuario	Se devuelve solo en S08 (`/me`) al dueño de la sesión; los logs usan el ID	Mientras la cuenta esté activa	Implementado. La cola de despacho no lo muestra: solo trae `cliente_id`
Correo	Inicio de sesión y contacto	Personal	auth	El propio usuario	Único; nunca en logs; la respuesta del login es igual si el correo no existe o la clave es incorrecta	Mientras la cuenta esté activa	Implementado
Dirección y comuna de despacho	Entregar el pedido	Personal	orders (copia dentro del pedido)	CLIENTE (la suya), DESPACHO (pedidos pagados), ADMIN	No aparece en logs ni en el reporte de ventas; DESPACHO no ve pedidos sin pagar	Igual que el pedido	Implementado en S11 (`despacho`)
Teléfono de despacho	Contacto para la entrega	Personal	orders (copia dentro del pedido, opcional)	CLIENTE (el suyo), DESPACHO (pedidos pagados), ADMIN	No aparece en logs; GraphQL no lo expone (Despacho solo tiene dirección y comuna)	Igual que el pedido	Implementado

Dato	Finalidad	Clasificación	Custodio	Roles que acceden	Protección	Retención	Estado en la U2
Fecha de nacimiento y sexo	Los pide el Caso 19 (RF5)	Restringido	clients	CLIENTE (los suyos), ADMIN	No se usan en ninguna pantalla ni servicio	Mientras la cuenta esté activa	Implementado: se guardan en clients y ningún servicio los devuelve
Contraseña	Autenticación (RF12)	Secreto	auth	Nadie	Hash bcrypt costo 12; nunca en logs, respuestas ni capturas (`check.py` no muestra el hash)	Se reemplaza al cambiarla	Implementado para el seed y para el registro (S09 la manda a auth, que guarda solo el hash). El front ya no guarda claves
Sesión (cookie)	Mantener la sesión	Secreto	auth (guarda solo el hash SHA-256 del token) y gateway (pone la cookie)	El navegador del usuario	Cookie `session_token` `httponly`, `samesite=lax`, TTL de 900 s con índice TTL en Mongo, invalidada en el logout	Hasta que expira o se cierra	Implementado. `secure` queda en `false` por defecto porque el entorno local usa HTTP
Pedidos y líneas	Compra, despacho y reporte	Personal	orders	CLIENTE (los suyos), DESPACHO (pagados), ADMIN, DUENO (solo totales en S19)	Propiedad revisada en orders con la identidad del gateway; pedido ajeno = 404	A confirmar (plazo tributario)	Implementado
Boleta (número, neto, IVA, total)	Comprobante de la venta (RF8)	Personal	orders	Quien puede ver el pedido	Igual que el pedido; la numera el servidor	A confirmar (plazo tributario)	Implementada (R08 parcial: sin envío por correo)
Pago (monto, estado, referencia)	Confirmar el pago	Personal	payments	CLIENTE (el suyo)	Monto tomado de orders, nunca del navegador; solo payments marca PAGADO; `Idempotency-Key` evita pagos dobles	Igual que el pedido	Implementado con proveedor simulado
Secretos entre servicios	Que solo el servicio autorizado llame a cada operación	Secreto	Vault (los del gateway) y `.env` (los demás)	Solo el servicio de cada integración	Uno por integración; nunca en el repo, logs ni capturas; comparación de tiempo constante	Se rotan si se filtran	Implementado. Solo el gateway los lee de Vault (brecha, ver M6)
`request_id`	Unir pantalla, respuesta, log y base	Interno	gateway (lo genera o valida)	Todos	Formato acotado (8 a 64 caracteres); no contiene datos personales	Igual que los logs	Implementado

3. ¿Custodiamos secretos del cliente?

No. Chocomanía no recibe ni guarda datos de tarjetas ni de cuentas bancarias: S16 solo recibe el `pedido_id`. El pago lo resuelve el proveedor (en la U2, un doble de prueba que respeta el contrato) y payments solo guarda el monto, el estado y la referencia que devuelve el proveedor (`SIM-...`).

El único secreto del cliente que guarda el backend es su contraseña, y solo como hash bcrypt. La excepción es el registro local del front, que no pasa por el backend (ver brechas).

4. Minimización

- La fecha de nacimiento y el sexo se piden en el registro porque el Caso 19 los exige (RF5), pero ningún servicio los recibe ni los usa.

- El reporte de ventas (S19) solo trae cantidades y montos por producto: sin nombres, correos ni direcciones.
- La cola de despacho (S14) trae el `cliente_id`, la dirección, la comuna, el teléfono y las líneas del pedido; no trae RUN, correo ni nombre. Sí trae el total y la boleta, porque se devuelve el pedido completo (posible mejora: una vista reducida para DESPACHO).
- GraphQL (S18) no expone el teléfono, y `stock` solo se entrega a ADMIN (`FORBIDDEN_FIELD` para el resto).
- Los logs usan campos fijos: servicio, evento, `request_id`, método, ruta, estado, `user_id` y duración. Nunca correo, dirección, teléfono ni secretos. El gateway registra solo los nombres de las claves que lee de Vault.

5. Recorrido de un dato restringido: la dirección de despacho

1. **Captura:** el cliente la escribe en el checkout (`checkout.html`). Viaja al gateway en el mismo origen (`http://localhost:8000`) y de ahí a orders dentro de S11.
2. **Validación:** orders exige dirección de 3 a 120 caracteres y comuna de 2 a 60. Si falla, el 422 dice qué campo falló, sin repetir el valor.
3. **Almacenamiento:** orders la guarda dentro del pedido (`despacho`). Es una copia: si el cliente cambia de dirección, el pedido no cambia.
4. **Consulta:** la ven el dueño (S12 y S18, esta última sin teléfono), DESPACHO cuando el pedido está pagado (S14) y ADMIN.
5. **Lo que no pasa:** no aparece en los logs (E02 y E03 muestran las líneas completas de una compra y un pago), en el reporte de ventas (S19) ni en la respuesta de GraphQL con teléfono.
6. **Fin:** se conserva con el pedido por el plazo tributario de la boleta (a confirmar).

6. Verificación del correo

El Caso 19 pide comprobar con una API de mensajería que el correo existe y está activo (RF5, RF11). En la U2 esta verificación es **simulada** en `verificar.html`: se genera un código y se muestra en pantalla, sin enviar un correo real. Se declara como brecha en el informe.

7. Pruebas de seguridad

Detalle de cada una en `docs/protocolo-pruebas.md` y código en `tests/api/test_seguridad.py`. Se ejecutan con `docker compose run --rm pruebas` sobre la base reseteada. Resultado: 16 de 16 pruebas aprobadas (las 6 de esta tabla y 10 casos más), ficha en `evidencias/u2/pytest/ficha-E07.md`.

ID	Caso	Resultado esperado	Resultado obtenido	Evidencia
P01	Visitante anónimo consulta el catálogo (S01)	200 con la lista de productos	**Aprobada:** 200 sin sesión	E01, E07
P02	Operación protegida sin sesión (S11)	401	**Aprobada:** 401 con formato de error único. También 401 si el navegador manda una identidad falsa (`X-Authenticated-User`)	E07
P03	CLIENTE intenta una operación de ADMIN (S03)	403	**Aprobada:** 403 y el producto no se crea	E07
P04	CLIENTE consulta un pedido ajeno (S12)	404 sin datos del pedido	**Aprobada:** 404 `ORDER_NOT_FOUND` igual que un pedido inexistente; también 404 al pagarlo (S16) y `null` por GraphQL (S18)	E07
P05	Llamada directa a un servicio interno sin pasar por el gateway	Rechazada	**Aprobada:** inventory, orders y payments responden 403 sin el secreto o con uno inventado	E07
P06	El front manda un precio o estado alterado (S11)	El servidor lo ignora y usa el precio de inventory	**Aprobada:** precio 8990 de inventory, estado PENDIENTE_PAGO y cajero USR-0005 pese a lo enviado; el pago usa el monto de orders	E07

8. Riesgos y brechas de datos

Riesgo	Situación	Tratamiento
~~Registro local con datos personales y clave en texto plano en localStorage~~ **Resuelto (INC05):** el registro va a clients y auth; el front ya no guarda cuentas y borra las que hubieran quedado de versiones anteriores	—	Brecha declarada; se resuelve con clients (S09 y S10) en la U3
Cuentas de prueba visibles en el login	`login.html` muestra correos y claves de prueba, incluida la de ADMIN	Son ficticias y solo para la demo; se quitan antes de cualquier despliegue real
Mongo con el usuario raíz en todos los servicios	Un servicio comprometido podría leer las bases de los demás	Brecha declarada (M6); en la U3, un usuario por base con permisos mínimos
Vault en modo dev y solo para el gateway	Los demás servicios reciben sus secretos por variables de entorno	Brecha declarada (M6)
Cookie sin `secure`	El entorno local usa HTTP	Se activa con `COOKIE_SECURE=true` al usar HTTPS
Cola de despacho con montos	S14 devuelve el pedido completo	Mejora para la U3 (vista reducida)
Identidad falsa en los logs de una llamada rechazada	En P05, inventory, orders y payments registran el `user_id` del header falso aunque rechazan la llamada (403)	No da acceso a nada; en la U3, el log no debería tomar ese header si falta el secreto

M8. GraphQL Gateway

Fuente: `backend/graphql-gateway/main.py`, los cambios del PR #37 en orders, inventory, gateway y `docker-compose.yml`, `frontend/js/mis-pedidos.js` (PR #38) y las evidencias `evidencias/u2/graphql-agregacion/` y `evidencias/u2/graphql-error-parcial/`. Código en `main` (commit `21c7b64`).

1. Qué es y dónde está

Característica	Cómo está implementado
Servicio	`graphql-gateway`, con Strawberry sobre FastAPI
Privado	Solo está en la red privada (sin puerto publicado). El único que lo llama es el API Gateway, por la ruta S18 `POST /api/v1/graphql`
Quién puede llamar	El API Gateway valida la sesión (S18 es "autenticado") y reenvía la identidad. graphql-gateway exige su propio secreto (`graphql_shared_secret`); sin él responde 403, y sin identidad, 401
Agregación real	Junta **orders** (pedidos del cliente) con **inventory** (estado actual de cada producto)
Solo POST	GET deshabilitado y sin GraphiQL (`graphql_ide=None`, `allow_queries_via_get=False`)
Pantalla real	`frontend/mis-pedidos.html` usa la query `misPedidos`

2. Schema

Strawberry convierte los nombres de Python a camelCase (`mis_pedidos` → `misPedidos`).

```
type Query {
  "Pedidos del cliente que inició sesión, del más antiguo al más nuevo"
  misPedidos(pagina: Int! = 1, porPagina: Int! = 10): PaginaPedidos!
  "Un pedido propio por ID (un pedido ajeno responde null)"
  pedido(id: ID!): Pedido
}

type PaginaPedidos {
  items: [Pedido!]!
  pagina: Int!
  porPagina: Int!
  total: Int!
}

type Pedido {
  id: ID!
  estado: String!
  total: Int!
  creadoEn: String!
  boleta: Boleta
  despacho: Despacho
  lineas: [Linea!]!
}

type Linea {
  productoId: ID!
  "Nombre al momento de la compra"
  nombre: String!
  cantidad: Int!
  precioUnitario: Int!
  subtotal: Int!
  "Estado ACTUAL del producto en inventory (puede venir null con error parcial)"
  producto: Producto
```

```
  "Pedido al que pertenece la línea"
  pedido: Pedido!
}

type Producto {
  id: ID!
  nombre: String!
  categoria: String
  imagen: String
  disponible: Boolean!
  "Solo ADMIN. Para los demás roles devuelve null con error FORBIDDEN_FIELD"
  stock: Int
}

type Boleta {
  numero: String!
  fecha: String!
  neto: Int!
  iva: Int!
  total: Int!
}

type Despacho {
  direccion: String!
  comuna: String!
}
```

No hay mutations: GraphQL solo se usa para leer. Las escrituras (crear pedido, pagar, anular) siguen por REST (S11, S13, S16).

Minimización: el tipo `Despacho` no incluye el teléfono, y `Pedido` no expone `cliente_id`, el historial ni el `request_id` de orders.

3. Resolvers

Campo	Qué hace	Servicio que llama	Secreto
`Query.misPedidos`	Revisa que el rol sea CLIENTE y la paginación (`pagina ≥ 1`, `porPagina` entre 1 y 20). Pide `GET /pedidos` a orders, que devuelve solo los pedidos de ese cliente	orders	`GRAPHQL_ORDERS_SECRET`
`Query.pedido`	Revisa que el rol sea CLIENTE y pide `GET /pedidos/{id}`. Si orders responde 404 (ajeno o inexistente), devuelve `null`	orders	`GRAPHQL_ORDERS_SECRET`
`Pedido.lineas`	Arma las líneas con los datos que ya trajo orders (sin llamadas nuevas)	—	—
`Linea.producto`	Pide el producto al DataLoader. Si el producto ya no existe, devuelve `null`; el nombre y el precio de la compra siguen en la línea	inventory (por lote)	`GRAPHQL_INVENTORY_SECRET`
`Linea.pedido`	Devuelve el pedido al que pertenece la línea (sin llamadas nuevas)	—	—
`Producto.stock`	Solo para ADMIN; a cualquier otro rol le responde con el error `FORBIDDEN_FIELD`	—	—

Secretos de solo lectura: orders acepta `GRAPHQL_ORDERS_SECRET` solo en `GET /pedidos` y `GET /pedidos/{id}` (función `verify_lectura`); con ese secreto no se puede crear, anular ni cambiar el estado de un pedido. inventory acepta `GRAPHQL_INVENTORY_SECRET` solo en `POST /interno/productos/lote`.

4. Batching (sin llamadas repetidas por ítem)

`Linea.producto` no llama a inventory por cada línea: usa un **DataLoader**, creado de nuevo en cada solicitud. Junta todos los ID de productos de la consulta y hace **una sola** llamada a `POST /interno/productos/lote` (máximo 50 ID).

Evidencia (`evidencias/u2/graphql-agregacion/`, `request_id` `gql-mispedidos-0002`): dos pedidos con cuatro líneas en total generan **una** llamada a inventory (`batch_loaded`, `cantidad: 2` productos distintos).

Hora (UTC)	Servicio	Evento
04:02:28.405	auth	`POST /introspect` → 200
04:02:28.629	orders	`GET /pedidos` → 200
04:02:28.808	inventory	`POST /interno/productos/lote` → 200
04:02:28.810	graphql-gateway	`batch_loaded`, 2 productos
04:02:28.813	graphql-gateway	`POST /graphql` → 200
04:02:28.814	gateway	`POST /api/v1/graphql` → 200

5. Límites

Límite	Valor	Para qué
Profundidad máxima	6 (`GRAPHQL_MAX_DEPTH`)	El schema tiene un ciclo (`Linea.pedido` → `Pedido.lineas` → …); sin límite se podría pedir una consulta infinitamente anidada
Alias máximos	10 (`GRAPHQL_MAX_ALIASES`)	Evita pedir el mismo campo muchas veces con nombres distintos
Tokens máximos	1.000 (`GRAPHQL_MAX_TOKENS`)	Limita el tamaño de la consulta
Paginación	`porPagina` máximo 20	Evita traer todos los pedidos de una vez
Plazo	3 s hacia orders e inventory (`UPSTREAM_TIMEOUT_SECONDS`)	El gateway espera 5 s, así que el error dice qué servicio falló

Ejemplo de consulta que se rechaza por profundidad (8 niveles):

```
query {
  misPedidos {
    items { lineas { pedido { lineas { pedido { lineas { nombre } } } } } }
  }
}
```

La consulta se rechaza antes de llamar a orders o a inventory, y el error queda en el log como `graphql_error` con código `VALIDATION`.

6. Autorización

Nivel	Regla	Dónde se revisa
Sesión	S18 exige sesión vigente	API Gateway
Servicio	graphql-gateway exige su secreto propio	graphql-gateway
Rol	`misPedidos` y `pedido` son solo para CLIENTE; otro rol recibe el error `FORBIDDEN`	graphql-gateway
Recurso	Un cliente solo ve sus pedidos: orders filtra por `cliente_id` y responde 404 al pedido ajeno, que GraphQL devuelve como `null`	orders
Campo	`Producto.stock` solo para ADMIN; los demás reciben `null` con `FORBIDDEN_FIELD`	graphql-gateway

7. Errores parciales

Si un servicio falla, GraphQL devuelve lo que sí pudo obtener y explica lo que falta en `errors`, con un código en `extensions.code`. Nunca muestra datos incompletos como si estuvieran completos.

Situación	Respuesta
inventory caído	Pedidos completos y `producto: null` en cada línea, con un error `UPSTREAM_UNAVAILABLE` por línea en `errors`
inventory lento (> 3 s)	Igual, con `UPSTREAM_TIMEOUT`
inventory responde con error	Igual, con `UPSTREAM_ERROR`
orders caído o lento	Sin datos (`misPedidos` no puede responder) y el error `UPSTREAM_UNAVAILABLE` o `UPSTREAM_TIMEOUT`
Paginación inválida	Error `BAD_REQUEST`
Rol que no es CLIENTE	Error `FORBIDDEN`
Campo `stock` sin ser ADMIN	`stock: null` y error `FORBIDDEN_FIELD`
Consulta demasiado profunda, con muchos alias o muy grande	Rechazada antes de ejecutarse

Evidencia (`evidencias/u2/graphql-error-parcial/`, `request_id` `gql-inventory-caido-0002`): con inventory apagado, los dos pedidos se ven con su estado y sus líneas, `producto` viene en `null` y hay 4 errores `UPSTREAM_UNAVAILABLE` (uno por línea), cada uno con su `path`.

En el front: `mis-pedidos.js` revisa `errors`. Si hay datos y errores, muestra los pedidos con el aviso "Algunos datos de los productos no están disponibles en este momento" y el código de la solicitud. Si no hay datos, muestra el error.

8. Uso desde una pantalla real

`frontend/mis-pedidos.html` envía esta consulta al cargar:

```
query MisPedidos($pagina: Int!) {
  misPedidos(pagina: $pagina, porPagina: 10) {
    total
    items {
      id estado total creadoEn
      boleta { numero }
      lineas { nombre cantidad producto { disponible } }
    }
  }
}
```

Con el campo `disponible`, la pantalla marca "(hoy agotado)" en los productos que ya no tienen stock.

9. Mapa de clasificación de servicios

Clasificación	Servicios (M1)	Por qué
Consulta	S01, S02, S08, S10, S12, S14	Leen datos de un solo servicio
Transaccional	S03, S04, S05, S06, S07, S09, S11, S13, S15, S16	Cambian el estado de un servicio
Integración	S17	La llama un sistema externo (el proveedor de pagos)
Agregación	**S18**	**Junta datos de orders e inventory en una sola respuesta (este módulo)**
Analítica	S19	Calcula el reporte de ventas a partir de los pedidos (ver `docs/adr-analitica.md`)

10. Límites conocidos

1. `stock` **solo se puede mostrar denegado.** Las queries son solo para CLIENTE, así que un ADMIN no llega a ese campo: la autorización por campo se demuestra con el caso `FORBIDDEN_FIELD`, no con

el permitido.
2. **Introspección activa.** Strawberry permite consultar el schema (`__schema`). El riesgo es bajo, porque solo se llega con sesión y a través del gateway.
3. **Más de 50 productos distintos en una página.** La operación por lote de inventory acepta máximo 50 ID. Si una página de pedidos tuviera más productos distintos, todos los `producto` vendrían en `null` con `UPSTREAM_ERROR`. Con el catálogo actual (5 productos) no puede pasar.

M9. Caracterización de carga

Tipo de dato: las cifras de las secciones 1 y 2 son una **proyección**, no una medición. La sección 4 queda para los resultados medidos con k6.

Decisiones relacionadas: `docs/adr-grpc.md` y `docs/adr-analitica.md`.

1. Supuestos del negocio

Supuesto	Valor proyectado	De dónde sale
Tipo de negocio	Chocolatería pequeña con un local y venta online	Caso 19 adaptado (local pequeño, negocio familiar)
Zona de despacho	Radio de 3 km alrededor del local	Caso 19
Pedidos por día (normal)	Hasta 100	Proyección para un local pequeño
Pedidos por día (peak)	Hasta 500 (5 veces lo normal)	Fechas de alta demanda: San Valentín, Día de la Madre, Navidad
Visitas al catálogo por pedido	20	Por cada compra, muchas visitas solo miran
Horario de mayor uso	12:00 a 14:00 y 19:00 a 22:00	Proyección

2. Carga proyectada

Indicador	Normal	Peak
Consultas al catálogo por día (S01, S02)	2.000	10.000
Pedidos creados por día (S11)	100	500
Pagos por día (S16, S17)	100	500
Usuarios al mismo tiempo	20	100
Solicitudes por segundo en la hora punta	Menos de 5	Menos de 25

Lectura: es una carga baja. El sistema es sobre todo de lectura (el catálogo concentra la mayor parte del tráfico) y las operaciones que escriben (pedidos y pagos) son pocas. Esto pesa en las decisiones de gRPC y analítica.

3. Umbrales de rendimiento

Se fijan **antes** de medir. Si una medición no los cumple, se registra como incidencia y no se cambian los umbrales para que pase.

Operación	Indicador	Umbral
Catálogo (S01)	p95 de duración	Menor a 500 ms
Crear pedido (S11)	p95 de duración	Menor a 1.000 ms
Todas	Tasa de errores	Menor a 1%
Catálogo (S01)	Throughput	Al menos 25 solicitudes por segundo (el peak proyectado)
Todos los contenedores	CPU y memoria	Sin contenedores reiniciados durante la prueba

4. Prueba de carga con k6

Script: `tests/carga/catalogo.js`. Se ejecuta con el servicio `k6` de `docker-compose.yml`.

Escenario:

1. Subir de 0 a 20 usuarios virtuales en 30 segundos (carga normal).
2. Subir a 100 usuarios virtuales en 30 segundos y mantener 1 minuto (peak).
3. Bajar a 0 en 15 segundos.

Cada usuario virtual hace `GET /api/v1/productos` a través del gateway y espera 1 segundo entre solicitudes.

CPU y memoria: se midieron en una tercera ejecución, registrando `docker stats` cada segundo y medio mientras corría k6 (81 muestras, `evidencias/u2/k6-docker-stats.csv`).

Cómo reproducirlo (todo dentro de Docker, sin instalar k6):

1. `docker compose up -d --build`
2. `docker compose run --rm seed --reset` (estado inicial con el seed)
3. `docker compose run --rm k6` (guarda el resumen en `evidencias/u2/k6-catalogo.json`)

Resultados

Ficha: `evidencias/u2/ficha-E08.md`. Se ejecutó dos veces: la primera no cumplió los umbrales y se registró como incidencia (INC06); la segunda, con el arreglo, sí. Los umbrales no se cambiaron.

Indicador	Umbral	Primera ejecución (`739419c`)	Segunda ejecución (`3c8d960`)	¿Cumple?
p95 catálogo	< 500 ms	3.163 ms	19,2 ms	Sí (después del arreglo)
Throughput catálogo	≥ 25 req/s	22,2 req/s	65,0 req/s	Sí (después del arreglo)
Tasa de errores	< 1%	0 %	0 %	Sí
CPU máxima (gateway / inventory)	Sin reinicios	No medido	No medido	Sí (tercera ejecución: 87 % y 93 % de un núcleo)
Memoria máxima (gateway / inventory)	Sin reinicios	No medido	No medido	Sí (tercera ejecución: 57,5 MiB y 92,9 MiB, sin reinicios)

Tercera ejecución, con CPU y memoria (09-11-2026, equipo de Flo, mismo escenario y umbrales): p95 de 59,8 ms, 63,7 solicitudes por segundo y 0 % de errores (8.616 solicitudes). El p95 es más alto que en la segunda ejecución (19 ms) porque fue en otro equipo, pero cumple con holgura. Máximos por contenedor:

Contenedor	CPU máxima (100 % = un núcleo)	Memoria máxima
gateway	87 %	57,5 MiB
inventory	93 %	92,9 MiB
mongo	110 %	370 MiB

Ningún contenedor se reinició y ninguno se acercó al límite de memoria (2,76 GiB en ese equipo). gateway e inventory quedan cerca de un núcleo completo con 100 usuarios, así que con más carga el siguiente cuello de botella sería la CPU de esos dos servicios (uvicorn corre con un solo proceso). Archivos: `evidencias/u2/k6-catalogo-con-stats.json` y `evidencias/u2/k6-docker-stats.csv`.

Qué pasó en la primera ejecución: el gateway creaba un cliente HTTP nuevo (`httpx.AsyncClient`) en cada solicitud, y armar ese cliente cada vez saturaba su CPU. Con 100 usuarios, cada respuesta tardaba unos 2 segundos. Se cambió a un solo cliente compartido para todo el gateway (INC06) y el p95 bajó de 3,2 s a 19 ms.

Fecha de ejecución: 08-11-2026, 18:17 y 18:35 (America/Santiago) **Archivos:** `evidencias/u2/k6-catalogo-antes.json` y `evidencias/u2/k6-catalogo.json` **Equipo donde se ejecutó:** computador Dell con Windows y Docker Desktop (k6 dentro de Docker, en el mismo equipo que el stack)

5. Límites de esta caracterización

- Las cifras de las secciones 1 y 2 son supuestos, no datos reales de un negocio.
- La prueba corre en un computador personal con todos los servicios en Docker, y k6 corre en el mismo equipo. No representa un servidor de producción.
- Orders, payments y graphql-gateway todavía crean un cliente HTTP por solicitud, igual que el gateway antes de INC06. No afecta al catálogo, pero puede pesar si se someten a carga pedidos y pagos.
- Solo se prueba el catálogo. Pedidos y pagos no se someten a carga en la U2.

ADR: uso de gRPC entre servicios

Estado: Aceptada **Fecha:** 2 de noviembre de 2026 **Módulo:** M9

Contexto

Los servicios de Chocomanía (gateway, auth, inventory, orders, clients, payments y GraphQL Gateway) se comunican hoy por HTTP con JSON. La pauta pide decidir si conviene usar gRPC en alguna de esas comunicaciones.

gRPC sirve sobre todo cuando hay muchas llamadas internas por segundo, mensajes grandes, streaming o la necesidad de latencias muy bajas entre servicios.

Según `docs/m9-carga.md`:

- La carga proyectada es baja: menos de 25 solicitudes por segundo en el peak.
- El tráfico es sobre todo de lectura del catálogo.
- Las llamadas internas por operación son pocas (por ejemplo, orders consulta a inventory para el precio y el stock al crear un pedido).
- Los mensajes son chicos (un pedido tiene pocas líneas).

Decisión

No se usa gRPC. Todas las comunicaciones entre servicios siguen siendo HTTP con JSON.

Alternativas consideradas

Alternativa	Por qué no se eligió
gRPC entre orders e inventory	Es la llamada interna más frecuente, pero con la carga proyectada el ahorro de latencia no se notaría. Agrega archivos `.proto`, generación de código y otra forma de depurar
gRPC en todos los servicios internos	El mismo costo multiplicado por cada servicio, sin un problema de rendimiento que lo justifique
HTTP con JSON (elegida)	Ya funciona, se depura con el navegador, Postman y los logs, y el `X-Request-ID` se propaga igual en todos los servicios

Consecuencias

- **A favor:** un solo estilo de comunicación, más fácil de probar, documentar y defender.
- **En contra:** si la carga creciera mucho, la llamada orders → inventory sería la primera candidata a optimizar.

Cuándo revisar esta decisión

- Si una medición de k6 muestra que la llamada orders → inventory no cumple los umbrales de `docs/m9-carga.md`.
- Si se agregan funciones que necesiten streaming (por ejemplo, seguimiento del despacho en tiempo real).

ADR: analítica del reporte de ventas

Estado: Aceptada **Fecha:** 2 de noviembre de 2026 **Módulo:** M9

Contexto

El Caso 19 pide un reporte de ventas por período para el dueño (R09). La pauta pide decidir si la analítica va en un servicio aparte o no.

Según `docs/m9-carga.md`, el volumen es bajo: hasta 500 pedidos al día en el peak. El reporte lo consulta pocas veces el dueño o el administrador, no los clientes.

Los datos que necesita el reporte (pedidos, estados, montos y líneas) ya están en orders.

Decisión

El reporte de ventas es un endpoint dentro de orders, no un servicio aparte:

- Ruta: `GET /api/v1/reportes/ventas?desde=&hasta=` (S19 en `docs/m1-servicios.md`).
- Acceso: DUENO y ADMIN.
- Se calcula al momento de la consulta, directo desde la colección de pedidos.
- Solo cuenta pedidos en estado PAGADO, EN_DESPACHO o ENTREGADO.

Alternativas consideradas

Alternativa	Por qué no se eligió
Servicio de analítica aparte, con su propia base de datos	Habría que copiar los pedidos a otra base y mantenerla sincronizada. Con este volumen no hay beneficio y aparece el riesgo de que el reporte muestre datos atrasados
Reporte calculado en el front	El front no debe ver los pedidos de todos los clientes. Rompe la regla de propiedad
Endpoint dentro de orders (elegida)	Usa los datos que ya existen, sin copias, y la consulta es liviana con este volumen

Frescura de los datos

Como el reporte se calcula en cada consulta, una venta aparece en el reporte **apenas el pedido pasa a PAGADO**. No hay un intervalo de actualización. Esto se puede probar en la U3: hacer una compra desde el front y ver que el reporte la incluye de inmediato.

Consecuencias

- **A favor:** un servicio menos que levantar, documentar y probar; datos siempre al día.
- **En contra:** si el volumen creciera mucho, consultas de períodos largos podrían hacer más lento a orders. Por eso se declara como recorte en el informe.

Cuándo revisar esta decisión

- Si el reporte de un mes completo supera el umbral de 1.000 ms.
- Si se piden reportes más complejos (por ejemplo, comparación entre años o tendencias).

Matriz de requisitos: pantalla, servicio, prueba, resultado y evidencia

Une cada requisito del Caso 19 (R01 a R13, detalle en `docs/m1-servicios.md`) con la pantalla donde se usa, el servicio que lo atiende, la prueba que lo comprueba y su evidencia. Revisada contra `main` (bdd26ff) y actualizada con las pruebas del 08-11-2026 (E07 y E08) las capturas del front (E09) y los servicios del panel por API (E10).

Estados: Aprobada (hay evidencia), Pendiente (la prueba existe pero no se ha ejecutado), Brecha (no se construyó en la U2).

Requisito	Pantalla	Servicio	Prueba	Resultado	Evidencia
R01 El cliente arma un pedido con los productos disponibles	`index.html` (catálogo), `carrito.html`, `checkout.html`, `mis-pedidos.html`	S01, S11, S12, S18	P01, P04, P06, C01 a C05, C08, C09	Aprobada	E01, E02, E05, E06, E07, E08, E09
R02 El pedido se prepara solo después de confirmar el pago	`pago.html`	S16, S17, S14 (cola solo con pagados)	P06, C06, C07, C10	Aprobada	E03, E04, E07, E10
R03 El cliente puede anular una compra indicando el motivo	`mis-pedidos.html` (botón "Anular")	S13	E02	Aprobada	E02 (estado ANULADO, motivo y `stock_devuelto: true`)
R04 Las ventas web se asocian a la caja de un cajero virtual	No tiene pantalla (lo asigna orders)	S11	E02	Aprobada	E02 (`cajero_id` USR-0005)
R05 Registro de clientes con sus datos y validación del correo	`registro.html`, `verificar.html`	S09, S09V, S10	`test_clientes.py` (registro, duplicados, validación en servidor, perfil propio, acceso directo)	Aprobada (verificación del correo simulada, ver R11)	E07
R06 El registro lo puede hacer el cliente o el administrador	`registro.html` (solo el cliente, local)	—	—	Brecha	—
R07 Administrar la disponibilidad (stock) de cada producto	`index.html`, `producto.html` (lectura); administración solo por API (panel interno no está en la U2)	S01, S03, S04	P01, P03, C11	Aprobada	E01, E07, E10
R08 Boleta digital por cada venta, enviada al correo	`boleta.html`	S12, S16	E03, E04	Aprobada en parte: boleta numerada con neto e IVA; sin envío por correo	E03, E04
R09 Reporte de ventas por período para el dueño	Sin pantalla (panel interno no está en la U2); probado por API	S19	C13	Aprobada	E10
R10 Mantenedores de productos, usuarios y clientes	Sin pantalla (panel interno no está en la U2)	S03, S04, S05 (productos)	P03, C11	Productos: aprobada por API. Usuarios y clientes: brecha	E07, E10

Requisito	Pantalla	Servicio	Prueba	Resultado	Evidencia
R11 Validar el correo con una API de mensajería	`verificar.html` (código simulado)	—	—	Brecha	—
R12 Autenticación con acceso según el perfil	`login.html`, menú de todas las páginas	S06, S07, S08 y el gateway	P02, P03, P04, P05	Aprobada	E07
R13 Órdenes de despacho en el orden en que llegan	Sin pantalla (panel interno no está en la U2); probado por API	S14, S15	C10, C12	Aprobada	E10

Pruebas que faltan

Todas las pruebas previstas se ejecutaron (E01 a E10). Lo único sin evidencia es lo que no se construyó: R05, R06, R11 y los mantenedores de usuarios y clientes de R10 (brechas declaradas).

Protocolo de pruebas, Entrega 2

1. Objetivo

Definir cómo se ejecutan y registran las pruebas del backend de Chocomanía, para que cualquier integrante pueda repetirlas y obtener el mismo resultado.

2. Ambiente de prueba

Elemento	Valor
Cómo se levanta	`docker compose up -d` desde la raíz del repo
Reinicio de datos	`scripts/reset.sh`: borra usuarios, sesiones y productos y vuelve a cargar el seed. Necesita el stack arriba
Datos iniciales	`scripts/seed.py`: 6 usuarios (uno por rol del front, dos clientes) y 5 productos con ID fijos, todos ficticios. Corre dentro de Docker (`docker compose run --rm seed`); `reset.sh` lo ejecuta con `--reset`
Logs de una solicitud	`scripts/logs.sh <request_id>`
Verificación en la base	`docker compose run --rm check pedido PED-0001` (también `producto`, `usuario` y `pago`). No muestra hashes de claves ni sesiones
Pruebas automáticas	`docker compose run --rm pruebas`: corre `pytest` en `tests/api` dentro de Docker y guarda la salida en `evidencias/u2/pytest/`
Prueba de carga	`docker compose run --rm k6`: corre `tests/carga/catalogo.js` y guarda `evidencias/u2/k6-catalogo.json`
Crear el `.env` en Windows (sin openssl)	`docker run --rm -v "${PWD}:/repo" -w /repo python:3.12.7-slim-bookworm python scripts/generar-env.py`

3. Reglas

1. Antes de cada ronda de pruebas, con el stack arriba (`docker compose up -d`), se ejecuta `scripts/reset.sh`, que deja la base en el estado inicial y carga el seed.
2. Cada prueba registra el `X-Request-ID` de la respuesta, para poder buscarla en los logs con `scripts/logs.sh`.
3. Cada ejecución tiene una ficha en `evidencias/`, hecha con `evidencias/plantilla-ficha.md`, y se registra en `evidencias/indice.md`.
4. Si el resultado obtenido no coincide con el esperado, se registra en `docs/incidencias.md`.

4. Identificadores

Los requisitos (R), servicios (S) y flujos (F) están en `docs/m1-servicios.md`.

Prefijo	Qué es
P	Prueba (lo que se quiere comprobar)
C	Prueba de contrato de un servicio del M1
E	Evidencia (ficha y archivos de una ejecución)
INC	Incidencia

5. Pruebas de seguridad (M7)

ID	Qué se prueba	Resultado esperado	Resultado obtenido	Evidencia
P01	Visitante anónimo consulta el catálogo (S01)	200 con la lista de productos	**Aprobada:** 200 sin sesión	E01, E07
P02	Operación protegida sin sesión (S11)	401	**Aprobada:** 401 con formato de error único. También 401 si el navegador manda una identidad falsa (`X-Authenticated-User`)	E07
P03	CLIENTE intenta una operación de ADMIN (S03)	403	**Aprobada:** 403 y el producto no se crea	E07
P04	CLIENTE consulta un pedido ajeno (S12)	404 sin datos del pedido	**Aprobada:** 404 `ORDER_NOT_FOUND` igual que un pedido inexistente; también 404 al pagarlo (S16) y `null` por GraphQL (S18)	E07
P05	Llamada directa a un servicio interno sin pasar por el gateway	Rechazada	**Aprobada:** inventory, orders y payments responden 403 sin el secreto o con uno inventado	E07
P06	El front manda un precio o estado alterado (S11)	El servidor lo ignora y usa el precio de inventory	**Aprobada:** precio 8990 de inventory, estado PENDIENTE_PAGO y cajero USR-0005 pese a lo enviado; el pago usa el monto de orders	E07

6. Pruebas de contrato

ID	Servicio (M1)	Caso	Código esperado	Código obtenido	Evidencia
C01	S11	Mismo `Idempotency-Key` y mismo cuerpo dos veces	201 y el mismo pedido, stock descontado una vez	201 y mismo pedido	E07
C02	S11	Misma clave con otro cuerpo	409 `IDEMPOTENCY_CONFLICT`	409	E07
C03	S11	Sin `Idempotency-Key`	400 `IDEMPOTENCY_KEY_REQUIRED`	400	E07
C04	S11	Producto sin stock	409 `STOCK_INSUFICIENTE`	409	E07
C05	S11	Un producto válido y un ID mal formado (INC04)	400 `INVALID_ID` sin descontar stock	400 y stock igual	E07
C06	S16	Mismo `Idempotency-Key` dos veces	201 y el mismo pago	201 y mismo pago	E07
C07	S16	Pagar un pedido ya pagado con otra clave	409 `PEDIDO_NO_PAGABLE`	409	E07
C08	S18	Consulta con profundidad mayor a 6	Rechazada, sin datos	Error de profundidad máxima	E07
C10	S14, S15	DESPACHO ve la cola y lleva PED-0001 de PAGADO a EN_DESPACHO y a ENTREGADO	200 en las tres	200, 200 y 200	E10
C11	S03, S04, S05	ADMIN crea, edita y elimina un producto	201, 200 y 200	201, 200 y 200	E10
C12	S15	Salto inválido de PENDIENTE_PAGO a ENTREGADO	409 `TRANSICION_INVALIDA`	409	E10
C13	S19	Reporte de ventas como DUENO y como DESPACHO	200 y 403	200 y 403	E10
C09	S01	Carga de 100 usuarios (M9)	p95 < 500 ms, ≥ 25 req/s, errores < 1 %	19,2 ms, 65 req/s, 0 % (después de INC06)	E08 (incluye la corrida con `docker stats`)

7. Pruebas manuales (U2)

En la U2 las pruebas de interfaz se registran a mano. Cada una lleva su ficha con pasos, resultado y archivos de evidencia.

ID	Flujo	Rol	Resultado	Evidencia
P07	F01, F03, F04, F05: compra completa desde el navegador (catálogo, login, checkout, pago, boleta y mis pedidos), vista de celular y mis pedidos con inventory caído	CLIENTE	**Aprobada:** PED-0004 pagado con boleta B-000003; pantallas adaptadas al celular; con inventory caído los pedidos siguen visibles con un aviso	E09

Registro de incidencias

Cada incidencia usa el prefijo `INC` seguido de un número correlativo. El prefijo `INCR-` es solo para los incrementos de la bitácora.

ID	Fecha	Detectada en	Descripción	X-Request-ID	Estado	Resolución
INC-001	2026-11-06	Revisión del PR #33 (payments)	Si orders no respondía al confirmar el pago, el pago quedaba APROBADO y el pedido en PENDIENTE_PAGO para siempre	— (revisión de código)	Resuelta	`a1b1de1`: el pago se guarda con `pedido_sincronizado: false` y payments reintenta cada 15 s. Evidencia E04
INC-002	2026-11-07	Revisión de la corrección del PR #33	Dos avisos de pago simultáneos podían marcar dos veces el mismo pedido como pagado	— (revisión de código)	Resuelta	`a1b1de1`: orders reconoce el aviso repetido por `pago_id` con una actualización atómica; un segundo pago aprobado queda REVERTIDO
INC-003	2026-11-08	Revisión del PR #38 (front)	El parámetro `volver` del login aceptaba cualquier URL: permitía `javascript:` (XSS) y redirigir a otro sitio	— (revisión de código)	Resuelta	`79588a7`: `login.js` solo acepta páginas propias (`nombre.html` con parámetros simples)
INC-004	2026-11-08	Revisión del código de inventory para el M1	En `/interno/reservas`, si un pedido trae un producto válido y luego uno con ID mal formado, se responde 400 sin devolver el stock ya descontado del primero	— (revisión de código)	Resuelta	`cffe475` (INCR-23): inventory valida todos los ID antes de descontar. `test_inc04_id_mal_formado_no_descuenta_stock` pasa (E07, `inc04-7853e5b1`)
INC-005	2026-11-08	Revisión del front para el M7	El registro local guarda los datos del cliente y su contraseña en texto plano en localStorage	—	**Resuelta** con el servicio clients (S09, S09V, S10): el front ya no guarda cuentas ni claves	Resuelta el 9 de noviembre con `backend/clients`
INC-006	2026-11-08	Prueba de carga del catálogo (k6, M9)	El catálogo no cumplió los umbrales: p95 de 3.163 ms (umbral 500 ms) y 22,2 solicitudes por segundo (umbral 25). Causa: el gateway creaba un `httpx.AsyncClient` nuevo en cada solicitud, lo que saturaba su CPU	— (prueba de carga, 3.019 solicitudes)	Resuelta	`3c8d960`: un solo cliente HTTP para todo el gateway. Nueva medición: p95 de 19,2 ms y 65 solicitudes por segundo, sin cambiar los umbrales (E08)
INC-007	2026-11-09	Revisión de la evidencia E10	En S15, si DESPACHO intenta mover un pedido sin pagar, recibe 409 con el estado actual ("No se puede pasar de PENDIENTE_PAGO a ENTREGADO"), aunque en S12 ese mismo pedido le responde 404. No expone datos del cliente ni montos	`5fe9266d56ba466f8f9f2102e1631604`	Declarada como brecha	En la U3, S15 debería responder 404 cuando el pedido no es visible para el rol (mismo criterio que S12)

Estados posibles: Abierta, En revisión, Resuelta, Declarada como brecha.